

Master's Thesis

A Scalable Computational Framework for Activity-Based
Dynamic Discrete Choice Models

M240492

AZWAN NAZAMUDDIN

Graduate School of Innovation and Practice for Smart Society
Hiroshima University

September 2026

ABSTRACT

Evaluating a transport policy honestly requires knowing not just an average travel-time saving but who gains, who loses, and by how much, as each person replans their whole day in response. An activity-based dynamic discrete choice model (DDCM) answers this in principle: it treats a day as one sequence of linked activity, timing, and mode choices, solved by backward induction so that the value of the best available day is itself a welfare measure. In practice, computing this value function exactly has been considered intractable at city scale: the reachable state space and the cost of a single backward induction pass both grow far beyond what a naive implementation can solve. Estimation, population-scale simulation, and policy evaluation all inherit this same barrier, since each reads from the same exact value function.

This thesis develops a computational framework that removes that barrier without approximating the model. States that a person cannot physically reach and return home from within a day are identified by a forward reachability search under the space-time prism and pruned before any solving begins, cutting a theoretical state space of the order of 10^8 to about 1.7 million reachable states, a 98.3% reduction, with no loss of exactness: pruned states carry zero probability by construction, so the value function and every choice probability are unchanged. The pruned graph is stored in compressed sparse form and solved by a GPU-parallel backward induction that exploits the graph’s inherent forward-time ordering, taking a full solve from the order of tens of hours to about a hundred seconds. This exact solver is embedded inside a nested fixed-point estimator with an analytical likelihood gradient from a second backward sweep, so maximum likelihood estimation costs the same order as one value-function solve, with no finite-difference noise. A synthetic recovery study confirms the estimator recovers known parameters with honest, well-calibrated standard errors, and a controlled comparison against sampling-of-alternatives and reinforcement-learning value approximations shows the exact solver is also the more accurate reference at equal or lower cost, not merely an affordable one.

The framework is demonstrated on Higashihiroshima City, Japan, a real 144-zone, five-mode network, estimating a schedule-preference activity-based DDCM from 7,390 weekday travel diaries. The model converges cleanly and reproduces the diary’s activity composition closely (McFadden $\rho^2 = 0.913$), though a single day of static-network data cannot separately identify the model’s earliness and lateness scheduling penalties, a limitation this thesis diagnoses structurally and addresses by fixing those penalties from the stated-preference literature for a policy application. Re-solving a 2-hour work-schedule delay for a sample of diary workers at the converged estimate shows a welfare gain for every worker in the sample, at every tested penalty value, an initially counter-intuitive result traced to how the fitted discretionary-activity timing profile reallocates non-work time across the day. The Higashihiroshima case study is evidence for that framework, not a second claim alongside it: it shows that exact, population-scale estimation, simulation, and the welfare analysis they support are all achievable on a real city network without sacrificing exactness for tractability.

Acknowledgements

I would like to express my sincere gratitude to Professor Makoto Chikaraishi for his invaluable guidance throughout this research.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	The Computational Barrier	2
1.3	Research Objective	3
1.4	Scope and Contributions	4
1.5	Thesis Structure	6
2	Background and Literature Review	7
2.1	Activity-Based Travel Demand Models	7
2.2	Dynamic Discrete Choice Models: Theory	8
2.3	DDCMs in Transport	9
2.3.1	Scheduling Preferences and Time-of-Day Utility	10
2.4	Approaches to the Computational Curse	11
2.4.1	Reachability	11
2.4.2	Graph Theory	12
2.4.3	Identification of Dynamic Discrete Choice Models	13
2.4.4	Approximating the Value Function: Sampling and Reinforcement Learning	13
2.5	Research Gap Summary	14
3	Model Specification	15
3.1	The Activity-Travel DDCM	15
3.2	Action Space	17
3.3	Transition Function	18
4	Scalable Computational Framework	20
4.1	The Computational Problem	20
4.2	Reachability-Based State Pruning	21
4.3	Sparse Graph Representation	23
4.4	GPU-Accelerated Backward Induction	25
4.5	Shared Universal Graph and Population Handling	27
4.6	Scalability	27
5	What the Framework Makes Possible: The Model on Higashihiroshima	29
5.1	The Higashihiroshima Data and Network	29
5.2	The Framework on This Network	31
5.3	Utility Function Specification	34
5.4	Numerical Experiments	38
5.5	Estimation Problem Setup	43
5.6	NFXP Implementation	44
5.7	Welfare and Policy Impact	45

6	Conclusion	50
6.1	Summary of Contributions	50
6.2	Limitations	51
6.3	Future Work	52
6.4	Concluding Remarks	53
A	Data Description	54
A.1	Person-Trip Survey	54
A.2	OD Level of Service	55
A.3	Zone Attractiveness	55
A.4	Mode Ownership and Choice	55

List of Figures

3.1	The daily schedule as a Markov decision process	15
3.2	Placing an observed trip onto the time steps	19
4.1	Why the state space is too large to enumerate	20
4.2	The algorithm at a glance	21
4.3	The space-time prism	22
4.4	Numerical verification that reachability pruning is lossless	24
4.5	Forward reachability over the day	24
4.6	Compressed sparse row layout, worked example	25
4.7	Level-synchronous backward induction on a GPU	26
4.8	Scalability on the synthetic network	28
5.1	The 144-zone study area	30
5.2	Accessibility on the case-study network	31
5.3	Population-scale simulation throughput	33
5.4	Heterogeneity scaling on a shared graph	33
5.5	Marginal-utility profiles and emergent activity timing	34
5.6	P_{open} distributions, real network vs. synthetic toy	36
5.7	The synthetic network for the experiments	39
5.8	Parameter recovery on the synthetic network	40
5.9	Profile log-likelihood, flat-home specification	41
5.10	Exact, sampling, and RL evaluation of the log-sum	42
5.11	Estimator sampling distributions: sampling vs RL	42
5.12	Baseline welfare by zone	46
5.13	Baseline welfare by zone, with and without a car	46
5.14	Mode and activity shares under the public-transport policy	48
5.15	Accessibility gain under the public-transport policy	49

List of Tables

3.1	State space dimensions and cardinalities	16
5.1	Computational framework performance	31
5.2	Utility parameters by component	38
5.3	Parameter recovery metrics, static-OD condition	40
5.4	Exact, sampling, and reinforcement-learning estimator quality . . .	43
5.5	Baseline weekday estimate	45
5.6	Welfare counterfactual	47
A.1	Person-trip survey sample sizes	54
A.2	Person-level demographics	54
A.3	OD level-of-service summary by mode	55
A.4	Zone attractiveness distribution	55
A.5	Weekday trip-leg mode shares	56

Chapter 1

Introduction

1.1 Motivation

Cities face persistent welfare trade-offs in transport policy. Congestion pricing, emissions controls, transit investment for ageing populations, and the integration of autonomous mobility all alter who can travel, when, and at what cost, yet the tools most widely used by planners cannot quantify these effects in welfare terms. The dominant four-step traffic model predicts aggregate vehicle volumes but treats mode and departure time as exogenous inputs, leaving the distributional consequences of policy invisible (Vickrey, 1969). Contemporary policy evaluations have made this gap tangible: the Stockholm congestion pricing trial generated welfare gains that were substantial and heterogeneous, but the aggregate traffic model used to justify the scheme could not recover them (Eliasson et al., 2009).

The difficulty is structural. Transport policies act through *when* people travel and *which trips they chain*, not through aggregate volume. Three reasons explain why static models cannot capture this. First, departure time is endogenous: a congestion charge that raises morning peak costs shifts departure time as well as mode, and a model that treats timing as fixed misses the primary behavioral response (Palma and Fosgerau, 2011). Second, the value attached to arriving on schedule or absorbing lateness costs is comparable in magnitude to the value of travel time itself (Small, 1982); any welfare calculation that ignores scheduling costs is materially incomplete. Third, time-varying policies such as peak-hour surcharges, evening-transit subsidies, and time-window restrictions have no static analogue, because the policy mechanism operates in the temporal dimension that static models collapse. Trip chaining compounds these problems: a child drop-off, a commute, a shopping stop, and a return pick-up are jointly determined and jointly affected by any policy that changes any one leg of the chain.

Dynamic Discrete Choice Models (DDCMs) for activity-based travel demand address both the behavioral and the welfare measurement problem within a single unified framework. By modeling each time step of the day as a sequential choice over destination, mode, and activity type, DDCMs generate activity-travel patterns as the outcome of forward-looking utility maximization under time-space constraints (Västberg et al., 2020). The log-sum of the value function at the start of the day, which equals the inclusive value of discrete choice theory, constitutes a welfare-consistent consumer surplus measure by the Williams–Daly–Zachary theorem (McFadden, 1978; Small and Rosen, 1981). Policy prediction and welfare measurement emerge from the same model, without additional assumptions. The model family continues to be developed actively: McCarthy et al. (2025) have recently extended activity duration-dependent utility within the SCAPER scheduling framework, demonstrating that behavioral realism and welfare consistency

can be improved simultaneously.

The central practical obstacle is computational. Computing the welfare measure exactly requires backward induction over every reachable state, and for a full city network the reachable state space is far too large for a naive implementation to solve on a single machine (Section 1.2 states the scale precisely). Every approximation that reduces this cost modifies the value function and therefore corrupts the welfare measure by an unknown amount. This thesis demonstrates that exact computation is tractable: reachability-based state pruning and GPU-accelerated backward induction together reduce the effective state space by more than 99 percent while preserving the value function exactly.

1.2 The Computational Barrier

The gap between the promise of activity-based DDCMs and their practical adoption in policy analysis is explained by a single barrier: evaluating the model exactly. Estimation inherits it by asking for that same evaluation repeated at every trial parameter; simulation and policy analysis inherit it directly.

Computational intractability

The state of the system at each decision step comprises six components: time of day, geographic zone, current activity type, elapsed episode duration, transport mode, and mandatory-activity completion history (Chapter 3). A naive implementation that tracked duration at full resolution would face a Cartesian product on the order of 10^8 theoretical states; backward induction must in principle visit every one. On a conventional processor the naive computation would require tens of hours and several terabytes of memory, rendering the model inoperable for any practical policy analysis cycle (Västberg et al., 2020).

This is a structural limitation, not a hardware one. The exponential growth of the state space with the number of state dimensions, a fundamental limitation known as the curse of dimensionality, cannot be overcome by faster processors alone (Powell, 2011). Approximations that reduce the state space modify the value function and therefore invalidate the welfare measure that motivates using DDCMs in the first place. Restricting computation to physically reachable states offers a principled path to an exact, welfare-valid solution without modifying the value function (Bansal et al., 2017; Oyama and Hato, 2019).

Why exactness is not optional

Updated upstream The computational barrier does not only make the model slow to evaluate; it also decides which route to the value function itself is available at all. Two established routes compute $\bar{V}(s; \theta)$, and each costs something real. The first exploits a property of the log-sum-exp recursion: exponentiating the value function turns it into a linear system, so the recursive logit of Fosgerau et al. (2013) solves $\bar{V}$ exactly as $(I - M(\theta))z = b(\theta)$, a closed-form matrix solve rather than an iterative sweep. Because M contains θ , this system must be rebuilt and re-solved at every candidate parameter vector, and at the scale of a full activity-scheduling state space, on the order of 10^8 states, the matrix alone would need on the order of 10^{16} entries: not buildable, let alone invertible. ===== Policy analysis requires not only that the model can be evaluated, but that its utility parameters θ , which govern activity preferences, scheduling sensitivity, and travel disutility, are recovered from observed travel diary data. In

the DDCM setting, the likelihood of any observed activity-travel path depends on the value function $\bar{V}(s; \theta)$, which requires backward induction at every candidate θ . The established estimation approaches each face limitations at city scale. The Nested Fixed Point algorithm (Rust, 1987) embeds full backward induction inside the maximum likelihood optimizer, achieving exact maximum likelihood at the cost of one DP solve per iteration, which is feasible only when each solve is fast. The MPEC reformulation (Su and Judd, 2012) avoids the nested structure but creates a constrained problem with as many constraint variables as states, infeasible at 10^6 scale. Conditional choice probability methods (Hotz and Miller, 1993) bypass the dynamic program entirely but require many observations per state, a condition that fails when a million states are visited by a few thousand respondents. Sampling approaches fix the value function at a reference parameter vector, reducing cost but introducing a frozen-value-function bias and losing exact maximum likelihood (Västberg et al., 2020). *~~~~~ Stashed changes*

The second route, taken by Västberg et al. (2020) for the case study this thesis compares against directly, samples a fixed number of complete day-paths per person instead and applies the McFadden (1978) correction that restores consistency. This scales because the full recursion is never re-solved, but the value function is fixed at a reference parameter vector instead of being resolved at each trial θ , so the correction is only as good as that reference, and the approach does not extend to a model with travel-time uncertainty. A third family sidesteps solving $\bar{V}$ exactly at all: value-function approximation by simulated or reinforcement-learning rollouts (Sutton and Barto, 2014) replaces the continuation value with a sampled estimate, which is biased at any finite budget unless the rollout itself is corrected, a bias this thesis quantifies directly against the exact reference in Chapter 5.

None of these is an independent problem to be solved on its own terms: each is the same computational barrier, viewed from a different angle. The first route runs out of capacity. The second and third trade capacity for approximation error. A framework that makes the exact recursion itself fast removes the reason to accept either compromise. Section 1.3 states the research objective that follows from this.

1.3 Research Objective

This thesis pursues a single objective:

- O1** Develop a scalable computational framework for exact-likelihood inference *~~~~~ Updated upstream in activity-based DDCMs, enabling exact backward induction on a full city network while preserving ===== in activity-based DDCMs, enabling city-scale backward induction while preserving ~~~~~ Stashed changes the welfare-consistent value function.*

The objective is narrow to state but broad in reach: if $\bar{V}$ can be swept exactly, at scale, everything built on top of it inherits that exactness. A nested-fixed-point search that re-solves $\bar{V}$ at every candidate θ becomes fast enough to actually run on a full city network, rather than a computation reserved for toy networks. Forward simulation reuses the same sweep, batched across an entire population, so population size stops being the bottleneck it is for a naive per-agent solve. Heterogeneity is handled the same way: one shared graph, many masked evaluation contexts (car ownership, work schedule, home zone), so a new context costs a re-sweep, not a rebuild. Policy and welfare counterfactuals read directly off $\bar{V}$,

with no separate approximation module to defend, because the value function they read from is the same exact one the estimator itself validates. Estimating the model on a real city network, the population-scale simulation built on that estimate, and the welfare counterfactual read off it are not a second objective alongside O1: they are the case study and results chapters’ demonstration that this reach is real, not a second goal judged on its own standing.

1.4 Scope and Contributions

The study focuses on Higashihiroshima City, Hiroshima Prefecture, Japan, partitioned into 144 traffic analysis zones. The Higashihiroshima person-trip travel survey provides a one-day travel diary for each respondent. Weekday and weekend are kept as separate samples, since weekday is work-anchored and weekend is dominated by home and discretionary activity, with **9,006** weekday and **8,751** weekend person-days built directly from the original survey records (Chapter 3). This thesis estimates and reports the **weekday** model; the weekend sample is held for future work (Section 6.3). The likelihood is evaluated on the person-days whose observed day can be represented as trips between zones. Computational hardware specifications are excluded from the body of the thesis per convention; the result that matters is that the pipeline runs on a single machine (Chapter 4).

This thesis makes three principal contributions, each aimed at a distinct facet of the same computational burden: how much of the state space has to be solved, how fast a solve runs, and how many times it has to be repeated across a population.

C1: Reachability-based exact state-space reduction

The daily activity-travel planning problem is reformulated as a finite-horizon optimization over a Directed Acyclic Graph (DAG) whose nodes are system states and whose edges are feasible state-action transitions. A forward breadth-first search from valid morning initial states then identifies and retains only those states reachable under the space-time prism constraints of Hägerstrand (1970): temporal feasibility, zone-to-zone travel time limits, mode-chain continuity rules, and mandatory-activity ordering. The two are one package, not two separate ideas: it is the DAG framing that makes the pruned graph solvable in a single backward sweep, and it is the pruning that makes that sweep affordable. The reduction cuts the theoretical state space by more than 99 percent while preserving exact optimality: every state reachable from a valid initial configuration is retained, and removed states are provably infeasible, so the value function and every choice probability are unchanged. Chapter 4 reports the full pruning results and the sparse graph representation this produces.

C2: GPU-parallel exact backward induction

Because the pruned graph has no cycles, backward induction over it proceeds level by level from the terminal time step to the initial state in a single non-iterative sweep, and every state at a given time step depends only on values already solved at later time steps. This makes the sweep parallelizable across an entire time layer at once, not just fast in sequence: a custom GPU scatter-reduce kernel solves a whole layer in one call, which Chapter 4 shows taking a full solve from the order of tens of hours to about a hundred seconds. This is what makes Barrier 1’s naive cost avoidable without approximating the value function.

C3: Amortized reuse across the population

All agents whose ordered mandatory-activity sequences share the same structure share a single DAG, so one backward induction computation covers an entire population group rather than one person. Per-agent cost falls from $O(N)$ to $O(\text{groups})$, and because the same shared graph, once built, is re-swept at every candidate θ without being rebuilt, the cost of estimating the model does not grow with how many people are in the sample, only with how many distinct groups they fall into (Section 4.6).

Applying the framework: estimation on a real city

Contributions C1–C3 together deliver Objective O1: a computational framework for exact-likelihood inference. Whether that framework actually works is a question only an application can answer, so this thesis embeds it in a Nested Fixed Point estimator (Rust, 1987), with the likelihood gradient computed via a second backward induction pass over the same DAG, and runs it on the full Higashihiroshima travel diary sample. The utility specification used for this (time-varying marginal utility profiles in place of hard activity time windows, grounded in the temporal utility profile framework of Supernak (1992) and Joh et al. (2003)) is the specification adopted for this case study, not a property the framework requires: C1–C3 place no restriction on the utility form beyond additive separability across time steps. The pipeline converges cleanly on the full Higashihiroshima sample, and the converged parameter estimate is reported, with its behavioral fit assessed, in Chapter 5.

iiiiii Updated upstream That chapter also characterizes what the diary data can and cannot identify. The earliness and lateness scheduling penalties, in particular, are not identified from a one-day diary on a static network, which is why the welfare counterfactual fixes them from the stated-preference literature rather than estimating them. This application, and the policy simulation built on it, are the evidence that Objective O1 is met, not a fourth contribution alongside C1–C3. =====

C4: Exact-likelihood NFXP with analytical gradient

Contributions C1, C2, and C3 together make it computationally feasible to embed full backward induction inside the maximum likelihood optimizer, following the Nested Fixed Point algorithm of Rust (1987) at city scale. The likelihood gradient is computed via a second backward induction pass over the same DAG, yielding the observed-minus-expected feature representation of the score function. Estimation on the Higashihiroshima travel diary data recovers behaviorally interpretable activity utility parameters. Chapter 5 characterizes the identification structure of the estimated model, including the likelihood ridge along the switching-cost dimension arising from the partial identifiability of trip-chaining costs from activity duration data alone.

Contributions C1, C2, and C3 together address Objective O1; Contribution C4 addresses Objective O2. An earlier exploration of Relational Markov Decision Processes as an abstraction for population heterogeneity was investigated and ultimately replaced by the universal-graph approach implemented in Chapter 4; a discussion is provided in Section 4.5 of that chapter. ~~~~~ Stashed changes

1.5 Thesis Structure

This chapter has set out the motivation for treating a day’s activities and trips as one connected plan, the computational barrier that has kept such models from real-network policy use, and the research objective and scope that follow from it. The remaining chapters are organised as follows. Chapter 2 reviews the activity-based and dynamic discrete choice literatures, traces the computational and estimation barriers that have kept full-day DDCMs out of practical policy analysis, and states the specific gap this thesis addresses. Chapter 3 specifies the activity-travel DDCM’s state and action spaces and its transition function, the structure Chapter 4 solves exactly and Chapter 5 instantiates with a specific utility form. Chapter 4 develops the computational framework that makes exact estimation of this model tractable, combining reachability-based state pruning with GPU-accelerated backward induction over the resulting sparse graph. Chapter 5 carries that framework onto the Higashihiroshima case study: it specifies and validates the utility function on a small synthetic network, where parameter recovery and the cost of approximation can be checked against a known truth, embeds the validated estimator in an exact NFXP fit to the real diary, reports the converged parameter estimates and their behavioural fit, and reads policy and welfare analysis directly off the resulting value function, with the one-day diary’s identification limit stated honestly rather than left implicit. Chapter 6 closes the thesis: it summarises the contributions against the computational barrier stated in Chapter 1, states the limitations the case study exposes, and sets out the welfare analysis and richer mode specification left for future work.

Chapter 2

Background and Literature Review

2.1 Activity-Based Travel Demand Models

Transport demand is derived demand: individuals travel not for the journey itself but to access activities such as work, education, shopping, recreation, and social obligations, distributed across geographic space and time. This observation has direct implications for model structure. The time at which a trip occurs, the sequence of activities it connects, and the mode chosen are jointly determined; a model that treats each trip independently cannot represent the intertemporal trade-offs that govern daily mobility.

The constraints that bind these decisions together were given spatial form by Hägerstraand (1970), whose space-time prism defines the set of locations reachable from a given starting point within a given time budget. The prism established that travel behavior is a trajectory through time-space, jointly constrained by the activities that must be performed, the locations where they can occur, and the transport system that connects them. This framing gave rise to the activity-based modeling tradition, which positions the daily activity program, rather than the individual trip, as the fundamental unit of analysis (Axhausen et al., 2002).

Adler and Ben-Akiva (1979) gave this joint requirement its first operational form: the full set of trips, activities, and modes an individual performs in a day, the daily travel pattern, is treated as the outcome of a single utility-maximization problem over all feasible patterns. The activity-based literature that follows this formulation does not depart from its goal so much as from its tractability, splitting along two lines depending on what is given up to keep the choice problem solvable.

Rule-based computational platforms, including TRANSIMS, ALBATROSS, FEATHERS, and TASHA, keep the full daily pattern but replace utility maximization with decision procedures derived from observed data and applied to a synthetic population (Rasouli and Timmermans, 2014). MATSim (Axhausen and ETH Zürich, 2016) is a more nuanced case: households are assumed to evaluate a daily plan's full utility, as in the DDCM tradition, but rather than solving for the optimal plan directly, a co-evolutionary algorithm perturbs and regenerates a small choice set of alternative plans over repeated simulation, and a multinomial logit model selects among them (Axhausen and ETH Zürich, 2016). Plan generation is where the heuristics sit, not the final choice, but the process does not mimic an actual within-day sequential decision, and no analytically derived value function results from it. Across all of these platforms, the same structural limitation holds: none produces a value function derived by backward induction, so none can compute a welfare-consistent consumer surplus measure from the model itself. Without a welfare measure, policy analysis reduces to pattern matching: predicting how the aggregate distribution of trips shifts without

quantifying the utility gained or lost by individuals.

Random-utility tour-based models keep the opposite trade-off. Bowman and Ben-Akiva (2001) extend the nested-logit framework to model destination and mode choice for a household’s primary tour together with conditionally dependent secondary tours, so every choice remains a maximum-likelihood-estimable, welfare-valid random utility model. What is given up is the joint treatment of time: departure time is either fixed exogenously or discretized into a few coarse periods, so the fine-grained timing a congestion charge acts on falls largely outside what the model can represent.

Dynamic Discrete Choice Models resolve both trade-offs at once. At each decision step the agent selects an action comprising a destination, mode, and activity type that maximizes expected utility over the remainder of the day, so every choice remains grounded in a random utility model estimable by maximum likelihood. The value function at the start of the day is the log-sum of utilities over all feasible schedules, which by the Williams–Daly–Zachary theorem equals Marshallian consumer surplus (McFadden, 1978; Small and Rosen, 1981), restoring the welfare measure rule-based platforms cannot provide. And because the decision step is defined at the resolution of the model’s own time discretization rather than at the level of a tour, the timing detail a congestion charge acts on is not sacrificed to keep the choice set tractable. Policy scenarios are evaluated as differences in this value function. Section 2.2 develops the formal DDCM framework, and Sections 2.3 and 2.4 survey how the framework has been applied and the barriers, computational and estimation, that have limited its adoption.

2.2 Dynamic Discrete Choice Models: Theory

The foundational model for dynamic discrete choice was introduced by Rust (1987) in the context of bus engine replacement decisions. A single agent faces a sequence of decision periods; in each period the agent observes a state $s \in \mathcal{S}$ and selects an action $a \in \mathcal{A}(s)$, receiving immediate utility $u(s, a; \theta)$ under structural parameters θ and transitioning to a next state s' according to a transition probability $p(s' | s, a)$. The agent maximizes expected discounted total utility. The Bellman equation expresses the recursive structure:

$$V(s; \theta) = \max_{a \in \mathcal{A}(s)} [u(s, a; \theta) + \beta \mathbb{E}_{s'} V(s'; \theta)], \quad (2.1)$$

where $\beta \in (0, 1]$ is the discount factor. The structural parameters θ are the objects of econometric interest: once recovered from observed choices, they characterize preferences and support welfare-consistent policy simulation.

Under the logit DDCM specification, the unobserved utility component ε_a is i.i.d. Type I Extreme Value across alternatives. The Gumbel distribution’s closure property then yields a closed-form integrated value function:

$$\bar{V}(s; \theta) = \log \sum_{a \in \mathcal{A}(s)} \exp(u(s, a; \theta) + \beta \bar{V}(s'; \theta)), \quad (2.2)$$

where $s' = \text{transition}(s, a)$ is the deterministic successor state and $\bar{V}$ denotes the integrated (expected maximum) value function. The multinomial logit choice probabilities follow immediately:

$$P(a | s; \theta) = \frac{\exp(u(s, a; \theta) + \beta \bar{V}(s'; \theta))}{\sum_{a'} \exp(u(s, a'; \theta) + \beta \bar{V}(s''; \theta))}. \quad (2.3)$$

The logit specification yields both a tractable value function and closed-form choice probabilities, making maximum likelihood estimation feasible once $\bar{V}$ is computed.

The welfare interpretation of $\bar{V}(s)$ underpins the policy-analysis motivation of this thesis. By the Williams–Daly–Zachary theorem, $\bar{V}(s)$ equals the expected maximum utility over all alternatives, the inclusive value of discrete choice theory (McFadden, 1978). Small and Rosen (1981) established that this inclusive value constitutes the correct consumer surplus measure in the static multinomial logit; the recursive extension to finite-horizon DDCMs maintains this interpretation level by level through backward induction (Fosgerau et al., 2013). Policy scenarios are thus evaluated as $\bar{V}^{\text{new}}(s_0) - \bar{V}^{\text{base}}(s_0)$, a compensating variation in utility units.

In the finite-horizon formulation used throughout this thesis, the planning horizon spans T decision steps with discount factor $\beta = 1$. The terminal condition enforces end-of-day home return: $\bar{V}(s, \tau = T) = 0$ for home-terminal states and $-\infty$ otherwise. Under the finite horizon, value iteration is replaced by a single non-iterative backward induction sweep from $t = T$ to $t = 0$: the value at each step depends only on the value at the next step, and the computation proceeds level by level. This property, standard in the recursive logit literature (Fosgerau et al., 2013), is the key structural feature that enables GPU parallelization in Chapter 4.

Parameter estimation follows the Nested Fixed Point (NFXP) approach of Rust (1987): an outer optimizer proposes candidate parameter vectors θ , and an inner backward induction computes the exact value function at each θ . The per-person log-likelihood of an observed trajectory $(s_0, a_0, s_1, a_1, \dots, s_{T-1})$ decomposes as

$$\ell_n(\theta) = \sum_{t=0}^{T-1} \log P(a_t \mid s_t; \theta), \quad (2.4)$$

where each log-probability is computed from the Q-values and value function produced by backward induction, and the analytical gradient follows from a second backward sweep of the same recursion, in the recursive logit framework of Fosgerau et al. (2013) (Chapter 5).

2.3 DDCMs in Transport

The recursive logit model of Fosgerau et al. (2013) extended the DDCM framework to network route choice. In the recursive logit, a traveler at node n selects a successor link to maximize expected downstream utility, and the value function satisfies the same log-sum-exp Bellman recursion as the general logit DDCM. The key insight of Fosgerau et al. (2013) was that a finite-horizon DDCM on a directed acyclic network is equivalent to a path-level multinomial logit over all feasible paths, with the log-sum of path utilities constituting both the value function and the network-wide consumer surplus. This path-MNL equivalence enables consistent maximum likelihood estimation without path sampling, and provides a clean welfare interpretation for network-level policies. The recursive logit has since been extended to pedestrian route choice, destination-choice sequences, and multi-stage trip itineraries.

The full-day activity scheduling DDCM of Västberg et al. (2020) applied the recursive logit framework to the complete daily activity-travel program. The model represents the 24-hour day as a finite-horizon sequential choice process in which individuals select destination, mode, and activity type at each 15-minute

decision step. The state space comprises six dimensions (time, zone, activity, duration, mode, and activity history), and the Cartesian product of realistic ranges yields on the order of one million reachable states on the Stockholm network. Two estimation approaches were proposed and evaluated. The first is exact NFXP: backward induction is re-executed at every optimizer iteration, guaranteeing exact maximum likelihood. The second samples R alternative full-day paths using a fixed reference value function and applies the McFadden sampling correction, reducing computational cost at the expense of exact likelihood. The Stockholm case study demonstrated that both approaches recover behaviorally plausible parameters, but reported that full-network deployment under exact NFXP would require approximately 1,000 CPU-days on national supercomputing infrastructure, establishing an explicit benchmark for the computational barrier that this thesis addresses.

McCarthy et al. (2025) extended the SCAPER scheduling model with piecewise-linear duration-dependent utility functions estimated from Swedish travel survey data. The duration-dependent specification allows the marginal utility of time spent at work, leisure, or home to vary across the episode, capturing satiation and scheduling pressure that a constant marginal utility cannot represent. The paper demonstrates that the DDCM framework accommodates richer preference structures and continues to improve behavioral realism; the computational barrier of backward induction at scale is acknowledged but not resolved in that work.

The logit DDCM also connects directly to Maximum Entropy Inverse Reinforcement Learning. Ermon et al. (2015) proved that for finite-horizon deterministic MDPs with unit temperature, the trajectory likelihoods of MaxEnt-IRL and logit DDC MLE are identical; the two estimation objectives are mathematically equivalent under these conditions (Ziebart et al., 2008). This positions the DDCM as a special case of entropy-regularized optimal control, connecting the transport modeling literature to the broader reinforcement learning and inverse RL traditions. The practical implication is that computational advances developed for reinforcement learning at scale are, in principle, applicable to DDC estimation; this connection motivates the PhD programme that this master’s thesis initiates, outlined in the concluding chapter.

2.3.1 Scheduling Preferences and Time-of-Day Utility

Two traditions describe how the value of an activity varies with the clock, and the thesis draws on both. The first is the scheduling-delay tradition of Vickrey (1969) and Small (1982), in which a traveller has a preferred time for an activity and pays a penalty for arriving early or late. In the standard form the cost of a trip is travel time valued at a rate α , plus a penalty β per unit of earliness and γ per unit of lateness around the preferred time, with the empirical ordering $\beta < \alpha < \gamma$ (Small, 1982; Palma and Fosgerau, 2011). This form is the workhorse of dynamic congestion modelling because it turns departure-time choice into a trade-off between travel delay and schedule delay. The second tradition replaces the preferred time and its penalties with a marginal-utility profile that varies smoothly over the day, so that the value of an activity is read directly from a time-of-day curve rather than from deviation around a point (Västberg et al., 2020; Supernak, 1992). The two are closely related: a sharp time-of-day curve approximates a schedule-delay penalty, and a flat one removes timing pressure altogether. This thesis adopts the scheduling-delay form for work and school, and the temporal-utility-profile form directly for the discretionary activities, where the time-of-day curve is supplied by measured venue opening probability rather

than a free spline (Chapter 3).

The synthetic experiments of Chapter 5 use a time-varying extension of the travel term to give departure time its own effect: mode utility is written as a function of departure time, $v_m(\ell \rightarrow d, t)$, framed as a congestion discomfort premium, in the sense already established by crowding-multiplier valuations of travel time, rather than a physical bottleneck delay. Departure time scales the marginal disutility of travel, not the travel time itself, so the clock advances by the same physical duration whether or not a trip falls in a peak window. The Higashihiroshima level-of-service matrix remains static, so this extension is used only on the synthetic network of Chapter 5.

2.4 Approaches to the Computational Curse

Approximations to the Bellman equation have been studied under the label of Approximate Dynamic Programming (Powell, 2011). The most widely applied approach represents the value function as a weighted sum of basis functions and updates the weights to minimize the Bellman residual. Linear value function approximation reduces storage and computation from $O(|\mathcal{S}|)$ to $O(K)$ for K basis functions, but introduces approximation error that compounds through backward induction. For the welfare applications motivating this thesis, where the key output is the value function at the initial state $\bar{V}(s_0)$, this error is uncontrolled and cannot be corrected after the fact. Neural network value function approximation offers greater representational flexibility but at a cost to structural interpretability: a learned value function cannot be attributed to identified preference parameters, and welfare differences between policies lose their economic interpretation.

2.4.1 Reachability

The most principled approach to reducing the state space without introducing approximation error is to restrict computation to states that are physically reachable from valid initial configurations. This idea has roots in forward reachability analysis from control systems: Bansal et al. (2017) review Hamilton-Jacobi reachability methods for safety verification, which characterize the set of states reachable under system dynamics from an initial set. Applied to discrete Markov decision processes, the analogous procedure is a breadth-first search forward from valid initial states, retaining only states reachable by feasible action sequences. States outside the reachable set have well-defined values (they would receive $-\infty$ in backward induction) but they never appear on any decision path and need not be stored or computed.

In transport, Oyama and Hato (2019) applied the space-time prism of Hägerstrand (1970) to restrict the path set of the recursive logit model, constructing a reachability-constrained network that excludes paths violating temporal feasibility. The approach improved behavioral realism and estimation stability by removing paths with excessive detour from the choice set, while retaining the Markovian structure and the path-MNL equivalence needed for consistent estimation.

The contribution of this thesis extends reachability-based pruning to the full activity-scheduling setting, where the state space is defined over time, zone, activity type, duration, mode, and activity history simultaneously. A level-synchronous breadth-first search identifies the exact reachable state set, the graph is stored in CSR format, and backward induction is parallelized across all states at the same time step on a GPU. Chapter 4 develops this approach and reports the resulting pruning performance by activity-sequence group.

2.4.2 Graph Theory

The reachable state-action set built in the previous section is, formally, a graph. Its nodes are states and its edges are feasible transitions, and computing the value function means visiting every node once, in an order that respects the time index. Because elapsed time is one of the state coordinates, this graph is a directed acyclic graph whose edges only point forward in time, the DAG formulation developed in Chapter 4 as Contribution C1. Correctness of backward induction requires every state reachable at time t to be resolved before any state at time $t + 1$ is touched, which is exactly what breadth-first search guarantees and depth-first search does not: a depth-first traversal can reach a state deep along one branch long before a shallower state on another branch is even discovered (Cormen et al., 2022; Sedgewick et al., 2009). The reachable graph must therefore be built and traversed level by level in time, not branch by branch.

Level-synchronous breadth-first search is well studied outside transport because it is exactly this kind of computation: at each level, every vertex in the current frontier can be expanded independently of every other, which makes the whole level a single parallel operation rather than a sequence of dependent steps. Merrill et al. (2012) showed that this independence lets a GPU expand frontiers of millions of vertices as batched, data-parallel operations rather than as a sequential queue, and Tithi et al. (2022) proved that a level-synchronous schedule achieves the theoretical lower bound on parallel running time regardless of graph shape. Berrendorf et al. (2014) and Buluc and Madduri (2011) extend the same idea to memory-bandwidth-limited and distributed settings respectively, and Besta et al. (2017) recast breadth-first search itself as a sparse-matrix, dense-vector product, the same computational primitive used by the scatter-based reduction that this thesis’s backward induction relies on (Chapter 4).

This thesis implements the level-synchronous sweep using a tensor-computation framework rather than hand-written parallel kernels. Tensor frameworks such as PyTorch were built to make exactly this kind of batched, data-parallel computation expressible as ordinary array operations, with the framework responsible for scheduling the work across whatever parallel hardware is available (Paszke et al., 2019). The approach is not specific to machine learning: Rush (2020) used the same abstraction to batch a family of dynamic-programming algorithms for structured prediction, and Zhao et al. (2026) reformulate stage-wise dynamic programming directly as masked tensor-broadcast operations, the same pattern this thesis uses for backward induction. Writing the computation this way trades some of the raw performance available to a hand-tuned kernel for portability across hardware, a trade that matters here because the target device has no vendor-specific alternative to compare against (Chapter 4).

Reachability itself has been formalized twice, in two literatures that rarely cite each other. In control theory, Hamilton-Jacobi reachability computes the set of states a dynamical system can reach or must avoid under bounded disturbances (Bansal et al., 2017), a formulation Doshi et al. (2022) extend to multiple start and terminal times at once. In graph theory, reachability indices such as Merz and Sanders (2014)’s pruned contraction hierarchy answer the same question, whether one node can reach another, for graphs too large to search from scratch on every query. The two traditions converge on the same principle for different reasons, which is evidence that restricting computation to the reachable set is not a transport-specific trick but a general property of sequential decision problems. A related idea, not pursued here, is to parallelize backward induction along the time axis itself rather than across states within a time step: Särkkä and García-

Fernández (2022) show that a dynamic program can be solved in logarithmic rather than linear time using a parallel scan, with an extension to the continuous-time Hamilton-Jacobi-Bellman setting (Särkkä and García-Fernández, 2024).

Two prior applications of GPU computing to activity-based travel demand bound this contribution from either side. Zhou et al. (2019) parallelize an activity-based demand model using hand-written CUDA kernels and report speedups up to a factor of fifty, demonstrating the potential of GPU computation for this model class without addressing the value-function computation of a DDCM specifically. Saleem et al. (2018) apply location sampling with an importance-sampling correction to the SCAPER model of Västberg et al. (2020) to make large-scale simulation tractable, the approximate alternative to the exact graph traversal developed here. That graph traversal and dynamic programming are related is not only an implementation convenience: Dudzik and Veličković (2022) show, using tools from category theory, that the message-passing computation of a graph neural network and the recursive structure of dynamic programming are the same computation, a further indication that the connection exploited in this thesis reflects the structure of the problem rather than a choice of software.

2.4.3 Identification of Dynamic Discrete Choice Models

Consistent estimation presumes the structural parameters are identified: distinct parameter vectors must imply distinct choice distributions. Magnac and Thesmar (2002) characterise the conditions under which the per-period utilities of a dynamic discrete choice model are nonparametrically identified from observed choices and transitions, showing that identification generally requires normalisations on a reference alternative and exclusion restrictions on the state. Their analysis motivates the validation strategy adopted in this thesis: rather than assert identification, we verify it numerically through a Monte-Carlo parameter-recovery study with profile-likelihood and coverage diagnostics (Chapter 5), and we adopt a single-transport-scale normalisation with the home activity as the reference, which removes a scale-versus-level ridge that otherwise leaves the travel-utility scale weakly identified.

2.4.4 Approximating the Value Function: Sampling and Reinforcement Learning

Two families of approximation make estimation scale by avoiding a full backward induction at every parameter trial. The first is *sampling of alternatives*: the choice-set log-sum is evaluated over a sampled subset of continuations with a correction term that restores consistency, following McFadden (1978) and, for multivariate extreme value models, Guevara and Ben-Akiva (2013); the closest analogue to the day-path sampling used here is the sampling-of-alternatives scheme for *route* choice of Frejinger et al. (2009). The second family estimates the value function by simulation: Monte-Carlo and off-policy value estimation (Sutton and Barto, 2014; Precup et al., 2000) replace exact backward induction with sampled rollouts, but without a correction the resulting plug-in likelihood is biased at any finite simulation budget. This distinction matters in the dynamic setting because the likelihood carries a *two-layer* error absent from static logit: in addition to sampling the choice-set log-sum — the single error layer analysed by Dekker (2025) for the static multinomial logit — each continuation value is itself produced by backward induction and may be approximated, adding a value-function-approximation layer. Chapter 5 uses the exact estimator as a reference

to quantify both layers, contrasting a corrected sampling estimator (consistent) with an uncorrected value-estimation estimator (inconsistent at finite budget).

2.5 Research Gap Summary

The literature leaves a specific gap. Activity-based DDCMs deliver the behavioural detail and the welfare measure that policy analysis needs (Section 2.1), but computing that welfare measure exactly requires backward induction over a state space too large to hold on a single machine, the barrier stated in full in Section 1.2. Every established route around it corrupts the welfare figure to buy tractability. Solving the recursive logit’s linear system exactly does not scale past a network-choice state space (Section 2.3); the case study this thesis compares against directly freezes the value function at a reference parameter vector and samples paths instead (Section 2.3); and approximating the value function itself, whether by a basis expansion or a learned network, trades the same exactness for a bias that cannot be recovered after the fact (Section 2.4). Reachability-based pruning is the one reduction that loses nothing, but it has been applied to network route choice rather than to the full activity-scheduling state space (Section 2.4.1). Realising that reduction at population scale is a graph-construction and graph-traversal problem, and the literature on parallel breadth-first search and tensor-based dynamic programming supplies a mechanism for it that has not yet been applied to activity scheduling (Section 2.4.2).

Finally, the utility tradition the welfare story rests on, scheduling preferences, carries an identification limit on one-day diary data that the literature has stated for congestion settings but not characterised for activity diaries (Section 2.3.1).

This thesis addresses this gap in Chapter 4, which makes exact backward induction tractable on the full activity-scheduling state space by combining reachability pruning with GPU-parallel backward induction, so the value function is preserved exactly without resorting to any of the three routes above. Chapter 5 then demonstrates that this is not only tractable but useful: the same exact machinery estimates and simulates on the real Higashihiroshima network, and a welfare counterfactual is read directly off the value function it produces, with the one-day diary’s scheduling-penalty identification limit stated honestly rather than papered over.

Chapter 3

Model Specification

3.1 The Activity-Travel DDCM

The daily activity-travel schedule is modelled as a discrete-time, finite-horizon Markov decision process (Rust, 1987; Västberg et al., 2020). The model is not tied to a particular city; it is instantiated on a case study, Higashihiroshima, one mid-sized city network, and the numbers below (144 zones, five modes) are its concrete instance rather than a property of the framework. The case-study network is partitioned into 144 traffic analysis zones with a full origin-destination level-of-service matrix covering travel time and monetary cost for five transport modes. Time is discretized into steps of $\Delta t = 15$ minutes, spanning the 24-hour day; the model horizon runs from midnight to midnight ($T = 1440$ minutes, $T/\Delta t = 96$ steps).

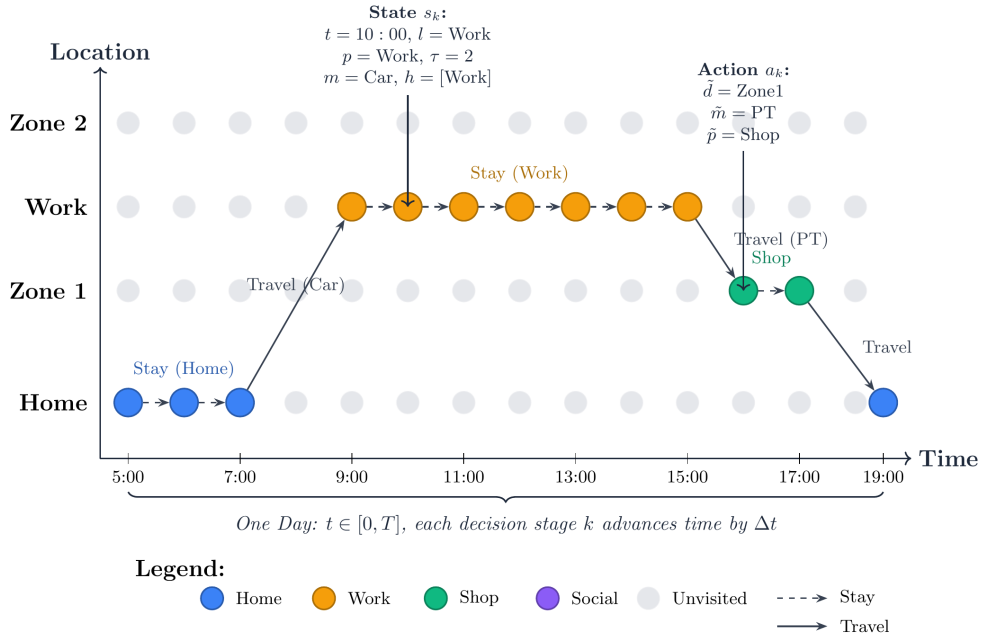


Figure 3.1: The daily schedule as a Markov decision process: a person moves through a space-time grid, time of day on the horizontal axis and location on the vertical. Most grid cells are never visited, which is what the reachability pruning of Chapter 4 exploits.

State vector

The system state at each step is the tuple

$$s = (\tau, z, a, d, m, h), \quad (3.1)$$

where the six components are defined as follows. (There is no separate “car in use” flag: whether the household car is currently out is encoded by the mode, which is non-empty while travelling by car and resets on return home.)

Table 3.1: State space dimensions and cardinalities, Higashihiroshima 144-zone network.

Symbol	Dimension	Values	Notes
τ	Time step	96	0, 15, ..., 1425 min from midnight
z	Current zone	144	CZONE_1 ... CZONE_144
a	Current activity	10	5 active; see below
d	Episode duration	2	tracked as $\{0, \geq 1\}$; see below
m	Transport mode	8	6 active; see below
h	Mandatory-activity history	3	Sequential index, $0 \leq h \leq k$
Cartesian product		$\approx 6.6\text{M}$	

The time index $\tau \in \{0, 15, \dots, 1425\}$ (minutes) identifies the current 15-minute interval of the day. Zone $z \in \{1, \dots, 144\}$ is the traffic analysis zone in which the agent currently resides or performs an activity. The activity variable a is drawn from ten diary-coded categories in the Higashihiroshima person-trip survey: Home, Travel, Work, School, Hospital, Shopping, Leisure, Sightseeing, pick-up/drop-off, and Other. Under the current specification, four destination activities are active: Home, Work (incorporating School at the data layer, with Sightseeing and Other merged into Leisure), Shopping, and Leisure. Travel is retained as a transitional state only. Hospital and pick-up/drop-off are disabled via constraint masks (Chapter 4).

Duration d records only whether the current activity or travel episode has just begun ($d = 0$) or has already been running for at least one step ($d \geq 1$). In the present specification the utility of an activity depends on the clock time τ but not on how long the activity has lasted (Section 5.3), so the counter never needs to tell the third step of an activity apart from the tenth. Its only job is to enforce the rule that an activity lasts at least one step, and two values are enough for that. A richer specification in which utility depends on elapsed time, such as a satiation term that lets the value of an activity fall the longer it runs, would need the counter to track its full range; the cost of that extension is discussed at the end of this section. Transport mode m is drawn from the diary-coded modes, of which six are active: car driver, car passenger, walking, bicycle, bus, and train. Motorcycle, moped, and taxi are excluded from the choice set. Car passenger is a distinct mode rather than a mere complement to car driver: the diary records the driver or passenger role directly (`driver_relationship`), so a person observed travelling by car without driving is encoded as a passenger.

The mode also carries the information that a separate car-availability flag would otherwise hold, which is why the state has no such flag. At home the mode is empty. Once a person leaves home as a car driver the mode stays “car” for the

whole excursion and resets only on arrival back home, so whether the household car is currently out can be read off the mode itself. The rule that a car trip must end back at home is enforced through the feasible mode transitions (Section 3.2), not through a separate state variable. Car passenger does not tie up the household car and is always available; car driver is available only while the car is at home.

The mandatory-activity history index $h \in \{0, 1, \dots, k\}$ tracks progress through the agent’s ordered sequence of k mandatory activities, following Västberg et al. (2020). This representation reduces the $O(2^k)$ binary history to $O(k+1)$ sequential states: $h = 0$ means no mandatory activity has yet been completed; $h = k$ means all k are complete. For the Higashihiroshima population $h \in \{0, 1, 2\}$ covers the full range of observed mandatory-activity sequences.

State space size

With the duration counter collapsed to two values, the Cartesian product over all dimensions is

$$96 \times 144 \times 10 \times 2 \times 8 \times 3 \approx 6.6 \text{ million} \quad (3.2)$$

theoretical states. A naive implementation that instead tracked duration at its full range, rather than collapsing it to two values, would face more than an order of magnitude more: about 110 million states, the order-of- 10^8 figure quoted in Chapter 4. Even six million is too many for standard backward induction once a separate graph is built for every person, so Chapter 4 restricts computation to the states an agent can actually reach given where the day starts and the time that remains. This prunes more than ninety-nine percent of the product and leaves on the order of one million reachable states, while preserving exact optimality.

What a duration-dependent utility would cost. The collapse to two duration values is the largest single saving in Equation 3.2, so it is worth stating what is given up if a later specification makes utility depend on elapsed time, as a satiation term would. Tracking the full duration range restores a factor of about sixteen (33 values rather than 2), taking the product back to the order of 10^8 . Two facts keep this within reach. The dominant reduction is reachability pruning rather than the duration collapse, and pruning applies to the larger space just as well, since states that differ only in an unreachable duration are still discarded. And duration need not be tracked at full fifteen-minute resolution: a coarse counter of six to eight hourly bins recovers most of the collapse while still letting an activity’s value decline with time spent. A direct measurement of the duration-augmented graph, with and without pruning, is left for a specification that needs it.

3.2 Action Space

At each state s , the agent selects an action $a = (z', m', a')$ consisting of a destination zone $z' \in \{1, \dots, 144\}$, a transport mode m' , and a next activity type a' (Västberg et al., 2020). Not all (z', m', a') triples are feasible from every state; feasibility is enforced by the following rules, which also define the edges of the reachable DAG constructed in Chapter 4.

1. **Duration minimum.** Each activity episode must last at least one time step ($d_{\min} = 1$) before the agent may depart; this prevents zero-duration episodes. The Travel state has no minimum duration; the agent transitions directly to

a destination activity upon arrival. Under the temporal-utility-profile specification, no maximum duration cap is imposed; activity termination emerges from the utility gradient crossing the home reservation floor (Section 5.3).

2. **Mode transitions.** The available modes at any state depend on the mode used to arrive. The transition rules are:
 - Car (driver): once a car trip begins, the agent must continue by car until returning home ($m' \in \{\text{car}\}$ while away from home).
 - Car passenger, bicycle, bus, and train: each may be followed by walking (e.g., last-mile on foot).
 - Walking: may transfer to bus or train.
 - From home (no active mode): all five active modes are available.
3. **Car availability.** A new car-driver departure is allowed only when the household car is at home, that is when the agent is at home with no active mode. The mode rule above then keeps the agent on “car” for the rest of the excursion, so the car returns home before it can be used again. Car passenger does not tie up the household car and is unrestricted.
4. **Activity-zone feasibility.** Home activities are only feasible at the agent’s home zone; Work activities are only feasible at the observed work zone. Shopping and Leisure are feasible at any zone with positive attractiveness. Disabled activities are infeasible at all zones and time steps.
5. **Mandatory-activity ordering.** The history index h advances by one when the agent completes the $(h+1)$ -th mandatory activity in the prescribed sequence. Mandatory activities out of sequence are blocked.
6. **Terminal constraint.** At $\tau = T = 1440$ the agent must be at the home zone performing the Home activity; states that violate this condition receive value $V = -\infty$ in the backward induction terminal condition.

No hard time-window constraints are imposed on work start times, shopping opening hours, or child-escort windows; timing is entirely governed by the utility gradient specification in Section 5.3. Chapter 4 shows how these feasibility rules are enforced as constraint masks during the GPU-accelerated BFS and backward induction.

3.3 Transition Function

Given a state $s = (\tau, z, a, d, m, h)$ and action $a' = (z', m', a'')$, the next state $s' = T(s, a')$ is determined deterministically by one of three cases.

In the first case, the agent stays in the current zone and continues the current activity ($z' = z, a'' = a, m' = \text{stationary}$):

$$s' = (\tau + \Delta t, z, a, d + 1, m, h).$$

Time advances by one step and the episode duration increments by one.

In the second case, the agent departs for a new zone ($z' \neq z, m' \neq \text{stationary}$), the activity transitions to Travel and the clock advances by the ceiling-rounded travel time:

$$\tau' = \left\lceil \frac{\tau + \text{TT}(z, z', m')}{\Delta t} \right\rceil \cdot \Delta t, \quad s' = (\tau', z', \text{Travel}, 0, m', h),$$

where m' is the chosen travel mode. Once it is car it stays car under the mode-transition rules until the agent returns home, which is how the state records that the household car is away (Section 3.2). The ceiling rounding ensures that transitions always land on a grid step.

In the third case, after arriving at a destination ($a = \text{Travel}$), the agent begins the intended activity:

$$s' = (\tau + \Delta t, z, a'', 1, m', h'),$$

where the mode m' is empty if z is the home zone (the car, if it was used, is now back home) and otherwise carries the travel mode just used, so the car stays accounted for while the agent is away. The history advances, $h' = h + 1$, if a'' is the next mandatory activity in the agent's sequence, and $h' = h$ otherwise.

If τ' exceeds the day horizon $T = 1440$ minutes in any of the above cases, the state transitions to the terminal state, which absorbs value $-\infty$ unless the agent is home-terminal, in which case it absorbs value 0. This is the boundary condition the backward induction of Chapter 4 sweeps from, formalized alongside the value function in Section 5.3 (Chapter 5).

Aligning observed trips with the time steps

To compute the likelihood, each observed day is compared against the model's choice probabilities, which are defined on the discrete 15-minute time steps. Every observed trip must therefore end at one of those time steps. The difficulty is that the duration a traveller reports in the diary usually differs from the travel time the network assigns to that origin, destination, and mode. Network car times in particular are free-flow and tend to be shorter than the times people report. If an arrival were placed at the reported minute, it could fall between two time steps the model uses, and that part of the day could not be matched to the model.

This is resolved in the standard way (Figure 3.2). The departure is kept at its observed time. The arrival is placed at the time step implied by the network travel time (rounded up to the next step). The few leftover minutes are then split between the end of the activity before the trip (t_D) and the start of the activity after it (t_P), each receiving half. What the traveller actually chose, namely which mode and which destination, is kept exactly; only the placement of the arrival within the 15-minute interval is adjusted to fit the time steps. Doing this recovers most of the car, walk, and bicycle trips that a naive alignment to the reported minute would have failed to match.

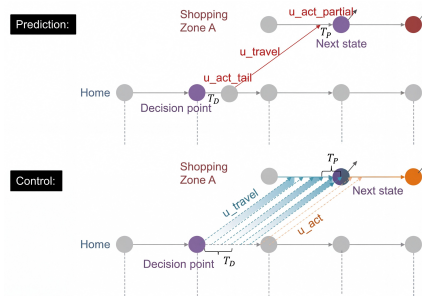


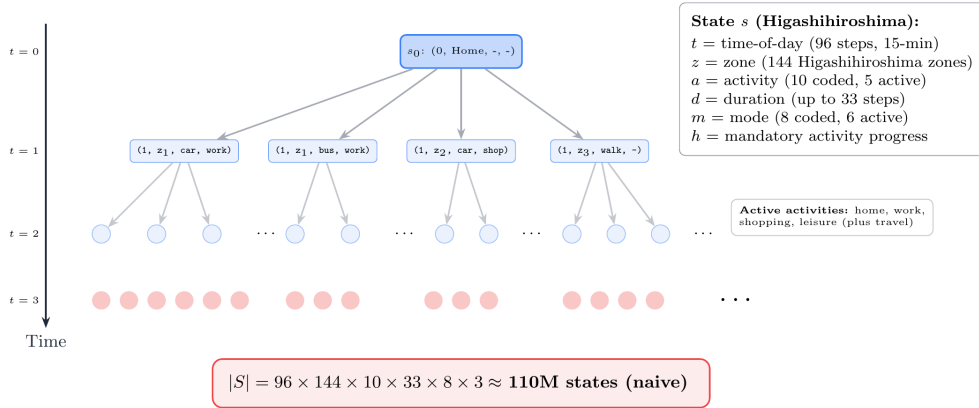
Figure 3.2: Placing an observed trip onto the 15-minute time steps. The departure stays at its observed time; the arrival is placed at the time step implied by the network travel time; the leftover minutes are split into a tail duration t_D on the prior activity and a partial duration t_P on the next activity. The lower panel shows an alternative that instead samples the leftover duration at random.

Chapter 4

Scalable Computational Framework

4.1 The Computational Problem

Chapter 3 defined the state as a six-component tuple and noted that, if the duration counter tracked its full range, the Cartesian product would run to the order of 10^8 states. Computing the welfare-consistent value function exactly means solving the backward induction of Equation (2.1) at every one of these states, and doing so afresh for every parameter vector the estimator proposes. A direct implementation fails on two fronts at once. The time cost is a full sweep over the state space per parameter trial, which on a single processor runs to tens of hours; the memory cost is a transition structure with on the order of 10^{12} entries if it is stored densely, far beyond what a single machine holds.



Higashihiroshima network; state representation after Västberg et al., Transportation Science, 2020

Figure 4.1: Why the state space is too large to enumerate: from the morning home state each step branches into every feasible action, and the branches multiply with the day. The state representation follows Västberg et al. (2020).

The way out is an observation about which states actually matter. Almost none of the theoretical states are ever visited on a feasible day. A person who starts the morning at home in one zone cannot be in a distant zone a few minutes later, cannot be at work before the network can carry them there, and cannot have completed a mandatory activity they have not yet reached. States that violate these limits have well-defined values in principle, but they lie on no feasible path, so they never enter the likelihood. Restricting computation to the states an agent can actually reach is therefore exact: it drops states that contribute nothing, not states that contribute a little. The rest of this chapter builds the machinery that

exploits this. Section 4.2 identifies the reachable set by forward search; Section 4.3 stores it compactly; Section 4.4 solves it in parallel on a GPU; and Section 4.5 shares one solve across an entire population group. Figure 4.2 lays out the full pipeline these pieces form, from the universal state space down to a per-person value function: pruning (rules P1–P4) removes what is physically or logically impossible for anyone and is done once, before any parameter is known; grouping by mandatory-agenda type and vehicle ownership (P5–P6) partitions the pruned space into a small number of shared topology graphs; masking then applies each person’s own home zone, work zone, and timing window to the shared graph to produce their evaluation context, the only step that depends on the parameter vector and is repeated at every optimiser iteration. The chapter walks through each stage in turn below.

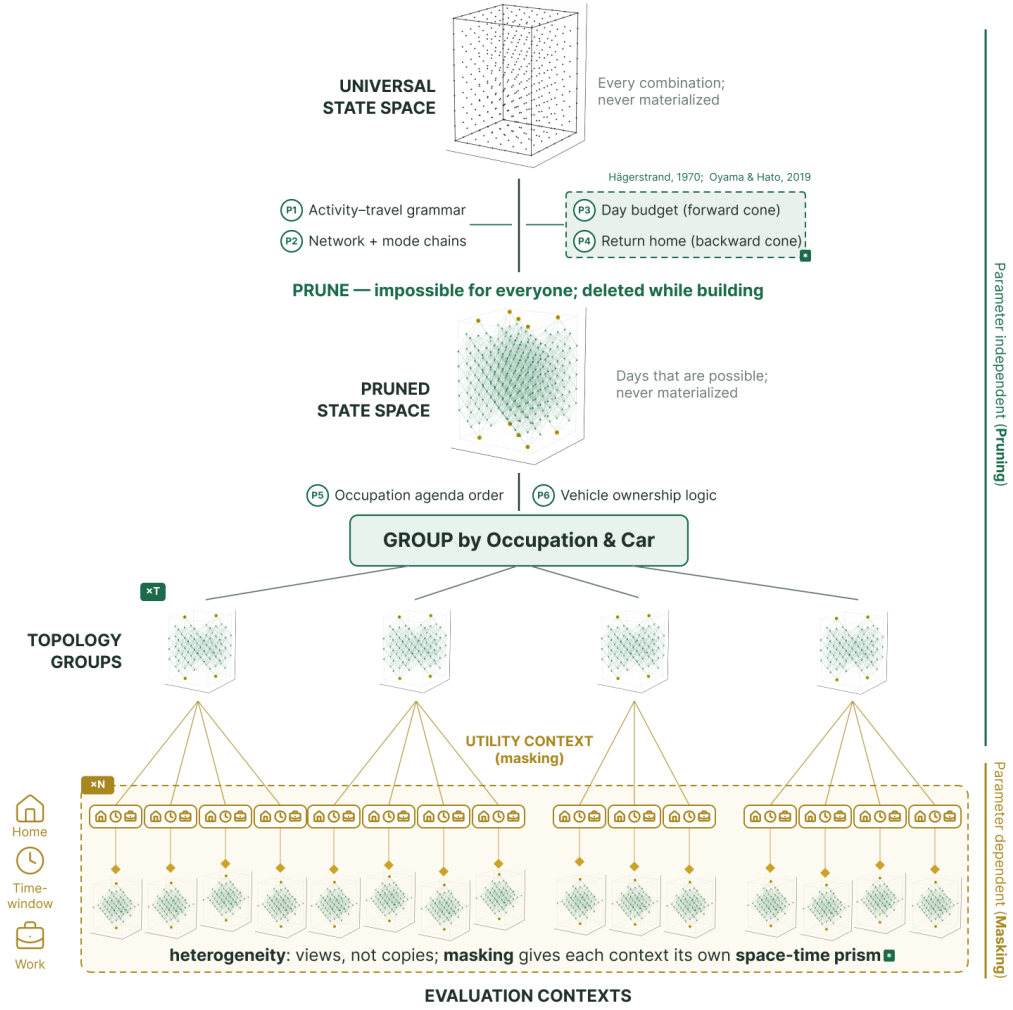


Figure 4.2: The algorithm at a glance: the computational pipeline from the universal state space to a per-person value function (pruning, grouping, masking), detailed in the paragraph above.

4.2 Reachability-Based State Pruning

The reachable set is found by a forward breadth-first search from the valid morning starting states. Beginning from each person’s home zone at the start of the day, the search repeatedly expands every state by every feasible action and collects the

successor states, advancing one time level at a time until the end of the day. A state enters the reachable set only if some feasible sequence of actions leads to it; everything else is left out. The feasibility rules are those of the space-time prism of Hägerstrand (1970) (Figure 4.3): an action is allowed only if there is time to perform it and still return home by the end of the day, only if the network can carry the agent between the two zones in the time available, only if it respects the mode-chaining rules of Section 3.2, and only if it advances the mandatory-activity sequence in order. Algorithm 1 states the procedure.

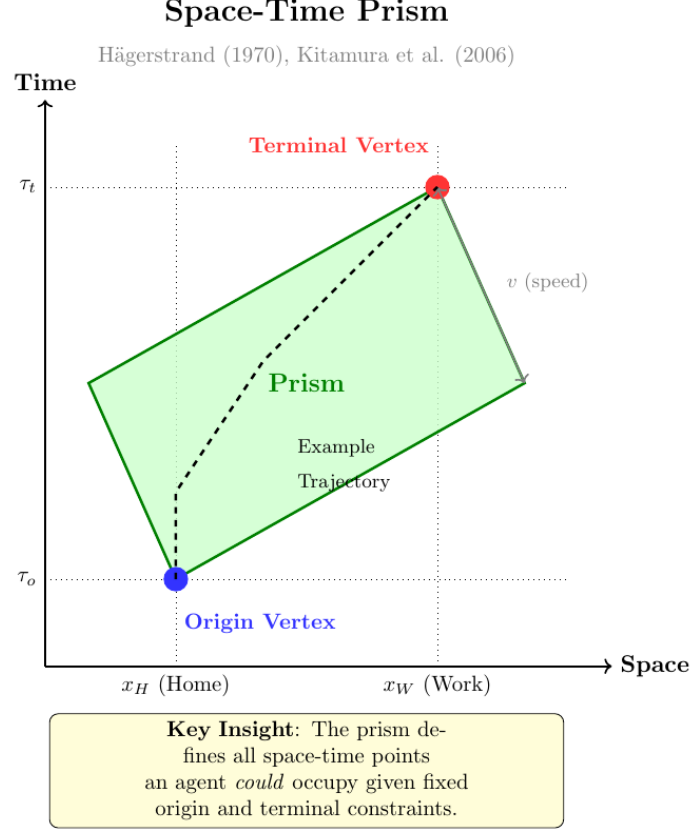


Figure 4.3: The space-time prism (Hägerstrand, 1970): the set of space-time points a traveller could occupy while starting and ending the day at home. In control-theoretic terms, the intersection of the forward-reachable and backward-reachable sets (Bansal et al., 2017).

Because the day has a fixed horizon and every action advances the clock, the search visits each time level exactly once and never returns to an earlier one, so it terminates in T passes and the states at a level can be expanded independently. Figure 4.5 shows the reachable set expanding outward from the home zone as travel becomes possible and contracting again as the end-of-day return constraint takes hold. On the Higashihiroshima network the search retains on the order of 1.5 million reachable states out of the 10^8 theoretical product, a reduction of more than ninety-nine percent, with a few hundred feasible actions leaving the average state. The reduction loses nothing: no state on any feasible day is removed.

Algorithm 1 Level-synchronous forward reachability

```
1:  $R_0 \leftarrow \{\text{valid morning home states}\}$ 
2: for  $\tau = 0, 1, \dots, T - 1$  do
3:    $R_{\tau+1} \leftarrow \emptyset$ 
4:   for all  $s \in R_\tau$  in parallel do
5:     for all  $a \in \mathcal{A}(s)$  satisfying the prism rules do
6:        $s' \leftarrow \text{transition}(s, a)$ 
7:       add  $s'$  to  $R_{\tau+1}$ ; record edge  $(s, a, s')$ 
8:     end for
9:   end for
10: end for
11: return  $\bigcup_\tau R_\tau$  and the recorded edges
```

Numerical verification of losslessness

The losslessness claim is checked directly rather than only asserted. Two checks are used, at the scale of the small synthetic network of Section 5.4, since only there can both sides of the comparison actually be built. The first toggles a single flag inside the production pipeline itself, disabling only the day-budget and return-home rules, the space-time prism refinement, and leaving every other feasibility rule, and the graph-construction code, untouched. Relaxing the refinement adds states to the graph, but every one of them turns out to already carry $V = -\infty$ under the remaining feasibility rules, so removing them changes nothing: $V(s_0)$ and the observed-path log-likelihood are bit-identical between the production graph and this relaxed alternative.

This first check leaves open whether the feasibility rules themselves, rather than only the prism refinement, are correctly enforced, since both builds still share the same graph-construction code. The second check removes that assumption: a separate implementation, sharing only the raw transition physics and reusing neither the breadth-first search nor the graph-construction code the production pipeline is built on, enumerates every state and action combinatorially and applies the same feasibility rules independently (Figure 4.4). Getting this second implementation to agree with production required finding five feasibility rules it had initially omitted, among them a minimum activity duration and a zone restriction on mandatory activities; each was found by tracing where the two implementations' value functions first diverged. Once corrected, the two independent implementations agree to floating-point precision, and the agreement holds not only at one network size but across every size tested, from three to nine zones, with the gap staying flat at that same precision rather than growing as the network does.

4.3 Sparse Graph Representation

The reachable states and the edges between them form a directed acyclic graph: nodes are states, edges are feasible state-action transitions, and every edge points forward in time. Storing this graph as a dense state-by-state matrix is out of the question. With 1.5 million states a dense transition matrix would hold on the order of 10^{12} entries, several terabytes in single precision, almost all of them zero because each state connects to only a few hundred successors.

The graph is stored in compressed sparse row form instead. A small set of flat arrays records, for every edge in time order, the index of its successor state

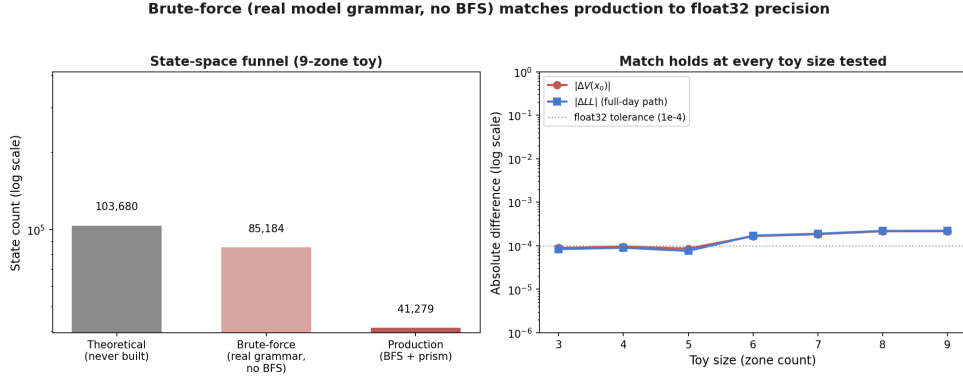


Figure 4.4: Numerical verification of the second check, on a nine-zone instance of the small synthetic network. Left: state-space size at three stages: the theoretical product (never built, 103,680), the from-scratch brute-force reference (real feasibility grammar, no breadth-first search, 85,184), and the production graph (breadth-first search plus the prism refinement, 41,279). Right: $|\Delta V(s_0)|$ and $|\Delta LL|$ (full-day path) between the two independent implementations, at every toy size tested from three to nine zones; both stay at or below the float32 tolerance of 10^{-4} .

Reachable state graph = space-time prism
forward cone $\wedge$ return-home cone $\rightarrow$ reachable states only (141 states, 792 transitions)

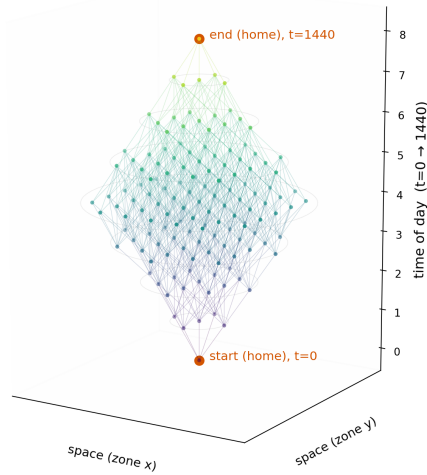


Figure 4.5: Forward reachability over the day. From the morning home state the reachable set expands as travel becomes possible and then contracts toward the end of the day, as the requirement to be home by the horizon removes states from which no legal return remains. Only the states inside this envelope are stored and solved.

and its immediate utility, together with an offset array that marks where each state’s outgoing edges begin. This keeps only the edges that exist. The same 1.5-million-state graph, with on the order of a few hundred million edges, occupies a few gigabytes, which fits in the memory of a single commodity GPU. The layout is also exactly what the backward induction of the next section needs: solving a state means reading a contiguous run of its outgoing edges, which the offset array locates in constant time. Figure 4.6 shows the layout on a four-state example. Algorithm 2 states the construction: a counting pass fixes how many edges leave

each state, a prefix sum over those counts gives the offset array, and a second pass writes every edge into the slot its source state's running cursor points to.

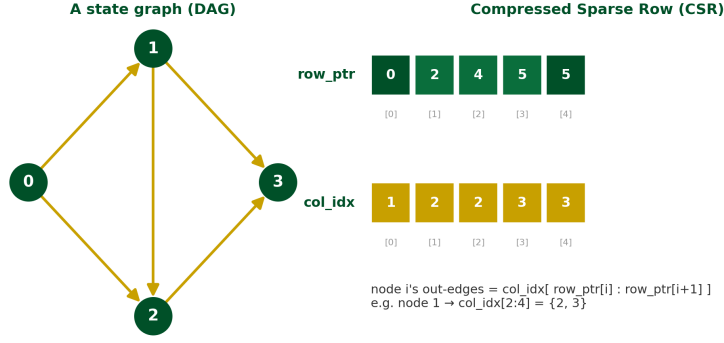


Figure 4.6: Compressed sparse row layout on a four-state example graph. `row_ptr` marks where each state's outgoing edges begin in `col_idx`; state 1's two out-edges are read off as `col_idx[row_ptr[1]:row_ptr[2]]`, a contiguous slice, with no search required.

Algorithm 2 Building the compressed sparse graph

Require: edges (s, a, s', u) from Algorithm 1

- 1: `count[s] ← 0` for every state s
 - 2: **for all** edges (s, a, s', u) **do**
 - 3: `count[s] ← count[s] + 1`
 - 4: **end for**
 - 5: `row_ptr[0] ← 0; row_ptr[s+1] ← row_ptr[s] + count[s]` for every $s \triangleright$ prefix sum
 - 6: `cursor ← row_ptr` (a working copy)
 - 7: **for all** edges (s, a, s', u) **do**
 - 8: `col_idx[cursor[s]] ← s'; edge_u[cursor[s]] ← u`
 - 9: `cursor[s] ← cursor[s] + 1`
 - 10: **end for**
 - 11: **return** `row_ptr, col_idx, edge_u`
-

4.4 GPU-Accelerated Backward Induction

With the graph in memory, the value function is computed by a single backward sweep from the end of the day to the start. The finite horizon makes this non-iterative: the value at the terminal step is fixed by the boundary condition (Section 2.2), and the value at every earlier step depends only on the step immediately after it. The sweep therefore visits each time level once, in reverse.

The work at each level is where the GPU helps. Within a level the states are independent, and the value of a state is a log-sum-exp over its outgoing edges (Equation (2.2)). This is a gather of the successor values, an addition of the edge utilities, and a segmented log-sum-exp reduction back onto the states, each of which runs in parallel across every edge and every state at that level. Algorithm 3 states the sweep, and Figure 4.7 illustrates the pattern: one level is solved at a time, but all states in it are solved at once.

The choice probabilities of Equation (2.3) follow from the same Q-values. Production runs use Apple Silicon GPUs through the Metal backend; the CUDA path in the code is inert on this hardware, and the CPU path serves only as a

Algorithm 3 Level-synchronous backward induction on the sparse graph

```
1:  $\bar{V}[s] \leftarrow 0$  for terminal home states,  $-\infty$  otherwise
2: for  $\tau = T - 1, T - 2, \dots, 0$  do
3:   for all edges  $e = (s, a, s')$  with  $s$  at level  $\tau$  in parallel do
4:      $Q[e] \leftarrow u[e] + \bar{V}[s']$ 
5:   end for
6:   for all states  $s$  at level  $\tau$  in parallel do
7:      $\bar{V}[s] \leftarrow \log \sum_{e \in \text{out}(s)} \exp Q[e]$ 
8:   end for
9: end for
10: return  $\bar{V}$ 
```

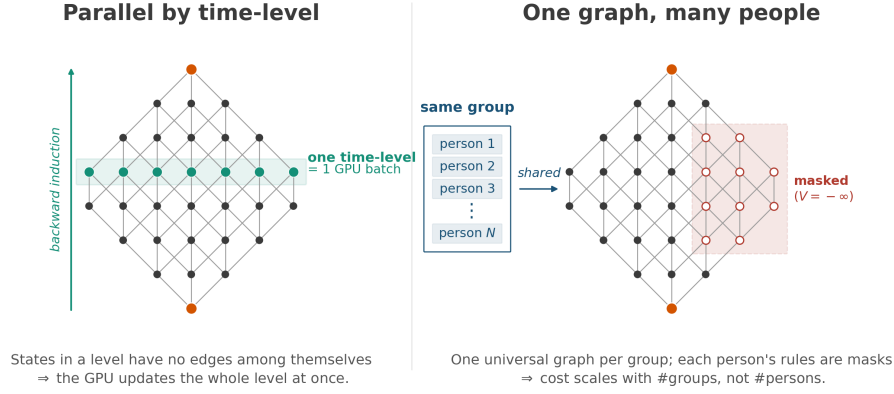


Figure 4.7: Backward induction proceeds level by level from the end of the day. All states at a level, and all edges leaving them, are processed in parallel; the only dependency is between adjacent levels, which the reverse-time order respects. The same graph serves every agent in a population group (Section 4.5).

fallback.

Analytical Likelihood Gradient

The same graph that carries the value sweep also carries its gradient. Differentiating the log-sum-exp recursion of Equation (2.2) term by term with respect to θ gives a second recursion of identical shape to Algorithm 3, computed by sweeping the graph once more in the same backward, level-synchronous order (Algorithm 4): at each edge the per-parameter feature gradient is weighted by that edge’s own choice probability under $\bar{V}$ and added to the continuation gradient already accumulated at its target, then summed into its source state exactly as the value itself was. This is the observed-minus-expected form of the score function used in Chapter 5 (Equation (5.8)), computed at the same asymptotic cost as one more backward-induction pass rather than by finite differences, which is why the optimizer of Chapter 5 never falls back on numerical differentiation. Automatic differentiation of the kind surveyed by Baydin et al. (2018) would in principle recover the same gradient by differentiating through the backward induction code directly, but at a memory cost proportional to the number of time steps, since every intermediate value has to be retained for the reverse pass. The analytical recursion instead computes the gradient with a second sweep of the same cost and memory footprint as the value computation itself, which is what makes gradient-based optimization of the full likelihood practical at this scale.

Algorithm 4 Level-synchronous analytical gradient sweep

Require: $\bar{V}$ from Algorithm 3; per-edge feature gradients $\nabla_{\theta} u[e]$

```
1:  $\nabla_{\theta} \bar{V}[s] \leftarrow 0$  for terminal home states
2: for  $\tau = T - 1, T - 2, \dots, 0$  do
3:   for all edges  $e = (s, a, s')$  with  $s$  at level  $\tau$  in parallel do
4:      $P[e] \leftarrow \exp(u[e] + \bar{V}[s'] - \bar{V}[s])$   $\triangleright$  edge choice probability
5:      $G[e] \leftarrow P[e] \cdot (\nabla_{\theta} u[e] + \nabla_{\theta} \bar{V}[s'])$ 
6:   end for
7:   for all states  $s$  at level  $\tau$  in parallel do
8:      $\nabla_{\theta} \bar{V}[s] \leftarrow \sum_{e \in \text{out}(s)} G[e]$ 
9:   end for
10: end for
11: return  $\nabla_{\theta} \bar{V}[s]$  for every state, in particular every group's initial state
```

4.5 Shared Universal Graph and Population Handling

A separate reachability graph for every individual would undo the saving: building and solving tens of thousands of graphs, one per respondent, returns the per-agent cost the framework set out to avoid. The key point is that the graph does not depend on the individual. Its nodes and edges are fixed by the feasibility rules, the network, and the ordered mandatory-activity sequence, none of which vary within a group of people who share that sequence. Two commuters with the same home-work-home structure traverse the same graph; they differ only in which zones their home and work happen to be, and that enters as a mask at solve time, not as a different graph.

One graph is therefore built per activity-sequence group, its backward induction is solved once, and every member of the group is then evaluated on the shared value function. In practice a small number of sequence groups cover an entire population, so the number of backward inductions per parameter trial is the number of groups, not the number of people. Individual constraints, the home zone, the work zone, and the observed timing, are applied as masks when each person's likelihood is read off the shared solve.

4.6 Scalability

The shared graph turns the cost of the method away from the number of people and toward the number of activity-sequence groups, which is small and fixed. The expensive steps, building the reachability graph and solving its backward induction, are done once per group; adding a person only adds a cheap forward pass on the already-solved value function. The per-person cost therefore falls as the population grows, whereas a naive implementation that re-solved the day for every person would pay the full solve every time.

Figure 4.8 measures this on the synthetic network of Section 5.4. As the population grows from a hundred to ten thousand agents, the total cost stays almost flat, because the one-time solve is amortised over more and more people, and the cost per agent falls toward the marginal cost of a single forward pass. A naive per-agent solve, by contrast, grows in direct proportion to the population and leaves the per-agent cost fixed and high; at ten thousand agents the shared approach is already several orders of magnitude cheaper per agent. This is what makes the exact estimator affordable at scale: the inner solve an optimiser repeats does not grow with the sample.

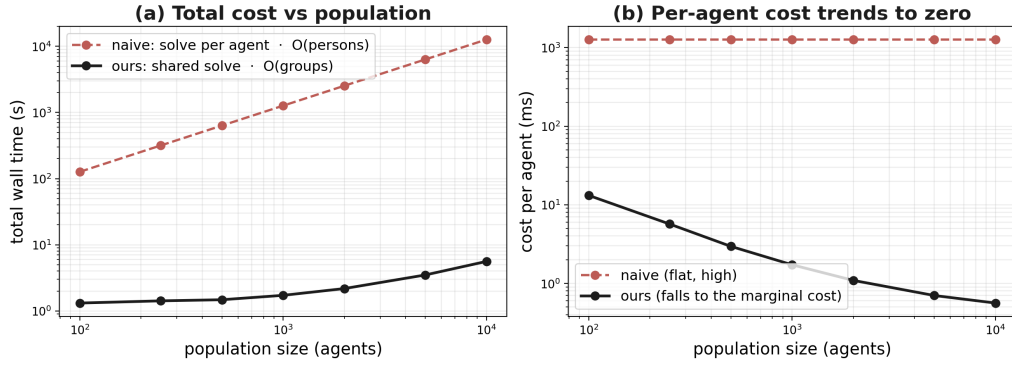


Figure 4.8: Scalability on the synthetic network, measured on a laptop CPU: (a) total wall time against population size, (b) cost per agent, shared graph vs. a naive per-agent solve.

The population-scaling argument above is only half of what amortization buys; the other half is the cost of the step an optimizer actually repeats: masking and solving one evaluation context. On the full Higashihiroshima weekday sample (7,376 valid persons), the universal graph resolves into four topology groups serving 259 evaluation contexts at sixty-minute schedule rounding. Building those four shared graphs, a one-time cost paid once regardless of how many parameter guesses or evaluation contexts follow, takes about 125 seconds total; masking and solving one evaluation context’s backward induction then takes 1.56 seconds on average. Four builds therefore serve all 7,376 people, about 1,844 people per build, the population-scaling argument above stated concretely on the real network rather than the synthetic one.

This is also where a scale-matched comparison against Västberg et al. (2020)’s own case study becomes possible. He reports 4–10 seconds per individual for his sampling-based evaluation, on a network of 1,240 zones against this thesis’s 144, an apparently unfair comparison in this framework’s favor on network size alone. It is not, once the state spaces are compared directly: his own enumeration, 65 time steps times 1,240 locations, about 80,600 states, is comparable to or smaller than one of this framework’s topology-group builds, $1.8\text{--}3.3 \times 10^5$ states each. The 1.56-second figure therefore solves a comparable-or-larger problem, exactly, faster than his approximate one. This is not because the network is smaller, but because of three things together: GPU-batched computation in place of a single CPU core, exact backward induction in place of sampling, and the amortized reuse just described, recomputed per individual in his method and shared across roughly 1,844 people per build in this one. Naming the three together matters: any one of them alone would not close the gap, and a bare speed number without this attribution would invite the wrong inference, that it is a hardware-neutral algorithmic result rather than a hardware-and-method combination.

Chapter 5

What the Framework Makes Possible: The Model on Higashihiroshima

The previous chapters developed the model and the computational framework without reference to any particular city. This chapter demonstrates what that framework makes possible on a real one, Higashihiroshima, along three lines. First, exact maximum-likelihood estimation becomes routine rather than aspirational once the value-function solve itself is fast (Section 5.2 measures the solve; Section 5.4 onward validates and then runs the estimator). Second, the same exact machinery scales to population sizes an estimation run never touches, because simulating a population reuses the value function rather than re-solving it (Section 5.2). Third, the converged value function supports policy analysis directly: baseline accessibility maps, a car-ownership comparison, and a scheduling-policy welfare counterfactual all read off the same $\bar{V}$ with no separate approximation module standing between the framework and the answer (Section 5.7). Higashihiroshima is a suitable proving ground for this: it comes with a one-day activity-travel diary and a complete origin-destination network, at a scale where the exact solve is affordable. The point of the chapter is the capability, not a claim about this city in particular. It is evidence for Objective O1 (Section 1.3), not a second objective that these results are judged against on their own.

5.1 The Higashihiroshima Data and Network

Person-trip survey

The behavioral data are the Higashihiroshima person-trip travel survey, a one-day activity-travel diary per respondent. The estimation sample is built directly from the original survey records (the full personal-info roster plus the weekday and weekend trip files), recovering respondents and behavioral fields that an earlier, stripped intermediate had dropped, in particular the driver or passenger role, driving licence, vehicle ownership, and parking use.

Weekday and weekend are kept as *separate* samples, since weekday is work-anchored and weekend is dominated by home and discretionary activity; this chapter reports the **weekday** model, with the weekend sample held for future work (Section 6.3). The sample also includes *stay-at-home* respondents (those who made no trips) as all-home person-days. A daily activity model needs the choice between going out and staying home, so excluding them would bias the model. This yields **9,006** weekday and **8,751** weekend person-days. The likelihood is evaluated on the subset whose observed day can be represented as trips between zones (see the same-zone note below), **7,390** weekday person-days.

OD level of service

The origin-destination level-of-service matrix covers all 144×144 zone pairs for the active modes, giving travel time and monetary cost per pair and mode. Public-transport entries that encode “no service” are treated as unavailable rather than as zero-time trips. The car travel times are free-flow network times; the travel times people report in the diary are systematically longer (for example, a car trip reported at about 22 minutes against a network time of about 15 minutes), which is why observed trips must be aligned to the model’s time steps (Section 3.3).

Zones, attractiveness, and accessibility

Zone attractiveness ($X[z]$ in Eq. 5.6) is the log-normalized retail/commercial floor area per zone (Figure 5.1b). As Figure 5.1a shows, the network resolves Higashihiroshima internally (B -code 1–12, 122 zones) and aggregates the rest of the prefecture into 22 coarse external zones (B -code 13). Trips out of the prefecture (a separate B -code) fall outside the network and are dropped.

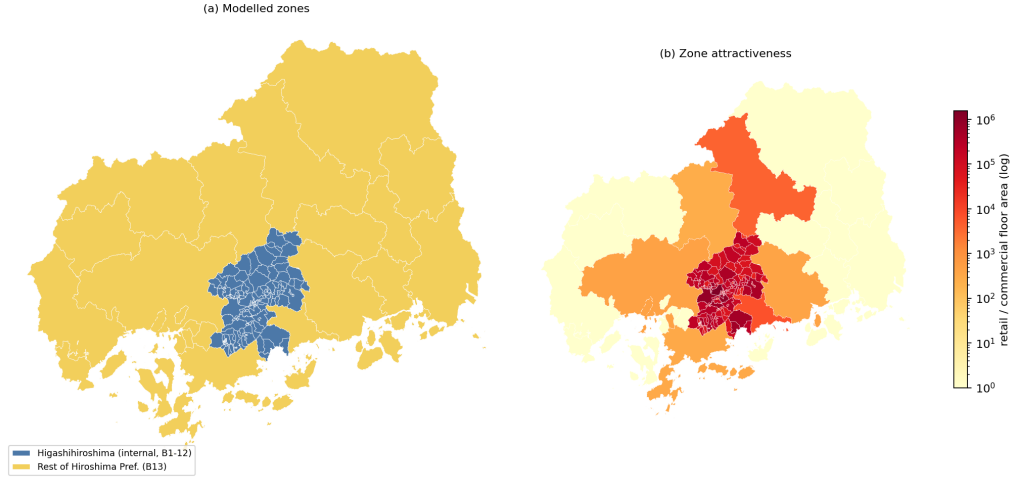


Figure 5.1: The 144-zone study area. (a) Higashihiroshima resolved internally (122 zones), the rest of the prefecture aggregated into 22 coarse zones. (b) Zone attractiveness (retail/commercial floor area, log scale); activity concentrates in the central Saijō area.

The modelled extent is a real but *mid-sized* city network. No claim of metropolitan scale is made: the contribution is that the value function is solved *exactly*, without sampling or approximation, on a full real-city zone system (Chapter 4); scaling the same machinery to larger metropolitan networks (e.g. the $\sim 1,300$ -zone Stockholm model of Västberg et al. (2020)) is left to future work.

Accessibility on this network is strongly centred (Figure 5.2): the central Saijō zone reaches the built-up core within minutes and the rural periphery only over an hour or more, and the number of zones reachable within half an hour by car falls from over a hundred in the centre to a handful at the edge. This structure is what the reachability pruning of Chapter 4 exploits: for any given person most of the network is simply too far to enter their feasible day.

Same-zone trips (scope). About 22% of observed road trips begin and end in the same zone. Because a zone has no internal road network, a same-zone trip has no well-defined travel time or change of location, so it cannot be represented as a

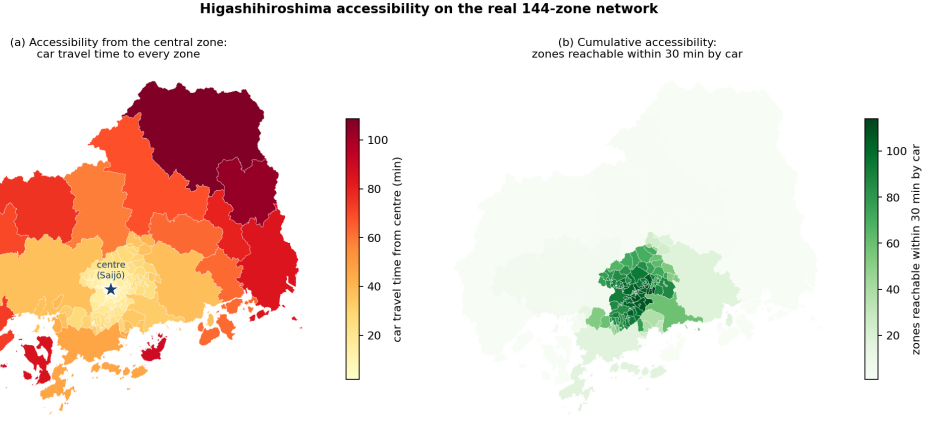


Figure 5.2: Accessibility on the case-study network. (a) Car travel time from the central zone (Saijō, star) to every zone. (b) Cumulative accessibility: zones reachable within 30 minutes by car.

movement in the model; such trips are excluded, and the likelihood is evaluated on trips between zones. This is a limitation of the zone resolution of the data, not of the framework. It is the same kind of boundary as the out-of-prefecture trips above, and it is stated explicitly rather than left implicit.

5.2 The Framework on This Network

Table 5.1 summarises what the framework buys on this network. Reachability pruning takes the state space from the order of 10^8 theoretical states to about 1.7 million reachable ones; the sparse layout takes the stored graph from terabytes to a few gigabytes; and the parallel backward induction takes a full solve from the order of tens of hours to about a hundred seconds, so that the inner solve an estimator repeats at every step becomes affordable. Per-agent simulation, used for the behavioural checks and policy runs, drops by a comparable factor.

Table 5.1: Computational framework performance on the Higashihiroshima network, from the production weekday estimation run (four shared graph topologies covering all 26 activity-sequence groups). Naive figures are for a single full-resolution solve without pruning, an estimate rather than a measurement, since that solve was never actually run to completion; framework figures are exact measurements from the run log.

Quantity	Naive	Framework	Factor
Theoretical / reachable states	$\sim 10^8$	1,672,243	98.3% reduction
Graph memory (CSR)	$\sim \text{TB}$	2.56 GB	$\sim 10^3 \times$
Graph build (one-time)	—	347 s	—
Full backward-induction solve	tens of hours	105 s/iter	$\sim 10^3 \times$
Per-agent simulation	$\sim 10^2$ ms	$\sim 10^0$ ms	$\sim 30 \times$

The reductions in the first three rows are preserved exactly: the value function the framework computes is the same one the naive solve would compute, to floating-point precision. The headline is not the size of the network but that the exact solve is affordable on a single machine. Higashihiroshima is a mid-sized city of 144 zones rather than a metropolis; the contribution is that its activity-scheduling DDCM is solved without approximation.

Population-scale simulation

The last row of Table 5.1 compounds when the framework is used as a population simulator rather than an estimator. Because the value function is solved per mandatory-sequence and home-zone group and shared by every agent in that group, the cost of simulating a synthetic population separates into a fixed part, the backward-induction solve, and a marginal part, the forward simulation of each agent. Algorithm 5 states the split: every group’s value function is solved once, Algorithm 3 from Chapter 4, and every agent afterward is stepped forward together, in one batched pass, sampling its next action from the same choice probability the value function defines (Equation (2.3)) rather than by any further search or re-solving. Here the synthetic population forms four sequence groups across the 144 home zones, so the framework builds four shared graph topologies and solves backward induction 576 times, once per sequence-and-home-zone group; those 576 solves are then reused across the entire population, whether it numbers one thousand agents or one hundred ninety thousand.

Algorithm 5 Batched forward simulation

Require: value functions $\{\bar{V}_g\}$ from Algorithm 3, one per sequence-and-home-zone group; a population partitioned into groups

- 1: **for all** groups g **in parallel do**
- 2: solve $\bar{V}_g$ once (Algorithm 3) ▷ shared by every agent in g
- 3: **end for**
- 4: $t \leftarrow 0$; $s_n \leftarrow$ agent n ’s morning home state, for every agent n
- 5: **while** $t < T$ **do**
- 6: **for all** agents n **in parallel do**
- 7: sample $a_n \sim P(a \mid s_n; \theta)$ using group $g(n)$ ’s value function $\bar{V}_{g(n)}$ ▷ Eq. (2.3)
- 8: $s_n \leftarrow \text{transition}(s_n, a_n)$
- 9: **end for**
- 10: $t \leftarrow t + 1$
- 11: **end while**
- 12: **return** simulated day-paths $\{s_n\}$ for every agent

Figure 5.3 measures this by simulating a synthetic population spread across all 144 zones at sizes from one thousand up to one hundred ninety thousand agents, the last of order Higashihiroshima’s resident population. The graph build and backward induction together stay in a 340–510 s band regardless of population size, since they depend on the topology groups the zones produce, not on how many agents are simulated; scaling the population $190\times$ instead raises the total wall time from about 350 to about 640 s, just over ten minutes on a laptop GPU. The marginal agent costs about 0.67 ms, and end-to-end throughput rises from a few to nearly three hundred agents per second as the fixed solve amortises. This is a throughput demonstration at reference parameters, not a behavioural result; its point is that once the exact value function is solved, simulating a city-sized population from it is cheap.

Heterogeneity scaling

The same shared-graph structure bounds the cost of the natural next modelling step, preference heterogeneity: a latent-class or random-coefficient extension in which several types share the state space but each carries its own value

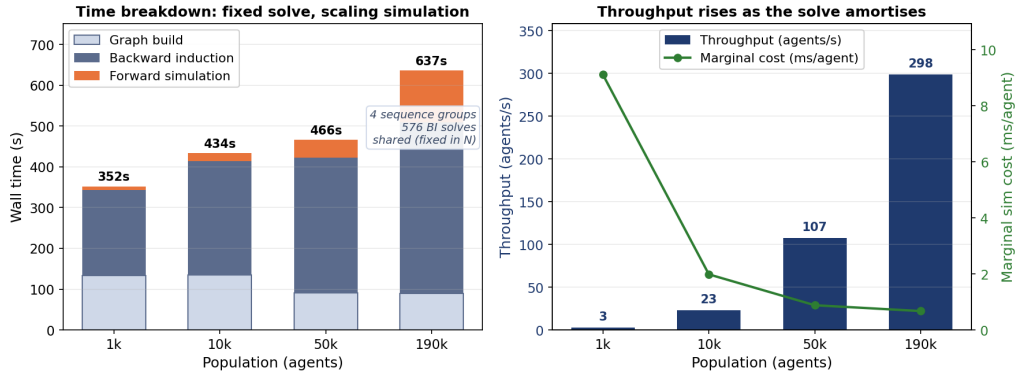


Figure 5.3: Population-scale simulation on the real 144-zone network. Left: wall-time breakdown by population size; graph build and backward induction are solved per sequence-and-home-zone group and are flat in the agent count, so total time is nearly constant while only the forward simulation grows. Right: end-to-end throughput (agents per second) rising as the fixed value-function solve amortises, and the marginal per-agent simulation cost falling below one millisecond. Synthetic population spread across all 144 zones, reference parameters.

function (Section 6.3). Because the graph topology does not depend on the utility parameters, adding a type is one more backward-induction solve on the same graph, not a new graph. Figure 5.4 measures this by solving the value function K times on one shared graph of about 828,000 states, for K from four to one hundred: the solve time is linear in the number of types at about 0.46 s each, so a hundred types solve in about 46 s after the one-time 27 s graph build. Each solve here varies the terminal home-zone condition rather than the parameters; a genuine latent class would also recompute the edge utilities from its own θ , a modest per-class addition that leaves the linear scaling intact. The framework therefore admits dozens of heterogeneous types at a cost that grows linearly, not combinatorially.

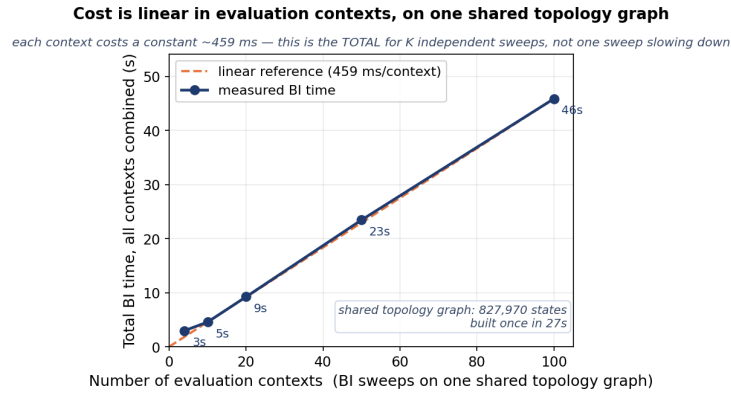


Figure 5.4: Backward-induction time against the number of distinct types (value functions) on one shared universal graph over the real 144-zone network ($\sim 828,000$ states, built once). Cost is linear in the number of types, about 0.46 s per type, so modelling dozens of heterogeneous preference classes is computationally feasible. Measured points track the linear reference; the small excess at low K is first-solve warm-up.

5.3 Utility Function Specification

The computational framework developed in Chapter 4 places one requirement on the utility function: additive separability across time steps, so that a backward induction sweep can accumulate it. Beyond that, it is agnostic to the specific form. The specification below is the one this thesis adopts for the Higashihiroshima case study, chosen and defended on its own behavioral merits, not derived from or required by the framework.

The model operationalizes the temporal utility profile framework of Supernak (1992) and Joh et al. (2003), in which each activity type generates a time-varying stream of marginal utility per unit time. An agent engages in an out-of-home activity only when its marginal utility profile exceeds the home alternative at that moment; no external scheduling rules are required.

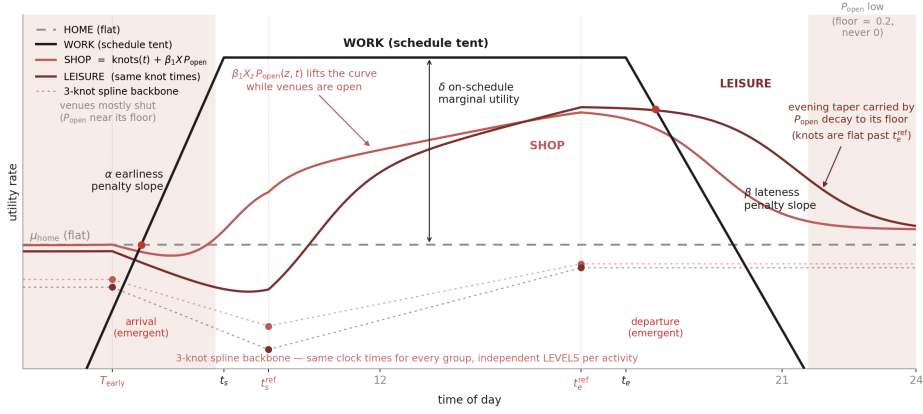


Figure 5.5: Marginal-utility profiles, conceptually, against the flat home reference μ_{home} (grey, dashed): work (black) follows the schedule tent of Equation 5.5, on-schedule rate δ between t_s and t_e with earliness slope α and lateness slope β outside it. Shop and leisure (light and dark red) follow Equation 5.6: the dotted three-knot spline backbone sets each segment’s level at the fixed clock times T_{early} , t_s^{ref} , and t_e^{ref} , and P_{open} lifts the curve while venues are open and tapers it toward its floor (≈ 0.2 , never zero) once they close. Arrival at and departure from work, marked on the axis, are not imposed but emerge where the work curve crosses the home line.

The work/scheduling component follows the Vickrey–Small α – β – γ scheduling form (Small, 1982; Palma and Fosgerau, 2011): work yields a flat on-schedule rate with linear earliness and lateness penalties around the individual’s preferred window. The mode component is specified under two treatments: a single transport *scale* (the fixed treatment), or freely estimated mode constants with time and cost coefficients (the free treatment). Only the fixed treatment is estimated in this thesis (Section 5.6); the free treatment is specified here but left for future work (Section 6.3).

Bellman recursion

Given the logit choice structure, the value function satisfies the log-sum-exp recursion (Fosgerau et al., 2013; Rust, 1987):

$$V(s) = \log \sum_a \exp(Q(s, a)), \quad (5.1)$$

$$Q(s, a) = u(s, a; \theta) + V(s'), \quad (5.2)$$

where $u(s, a; \theta)$ is the immediate utility of action a in state s , s' is the resulting next state, and the discount factor is $\gamma = 1$ over the full-day horizon. The terminal condition enforces end-of-day home return:

$$V(s, \tau = T) = \begin{cases} 0 & \text{if } s \text{ is a home-terminal state,} \\ -\infty & \text{otherwise.} \end{cases} \quad (5.3)$$

The value function at the initial state, $V(s_0)$, is the welfare-consistent expected total utility at the start of the day; policy scenarios are evaluated as $\Delta V(s_0) = V^{\text{new}}(s_0) - V^{\text{base}}(s_0)$.

Activity utilities

Home

Home yields a flat per-minute comfort value, the reservation floor that every out-of-home activity must exceed:

$$u_{\text{home}}(\tau) = \mu_{\text{home}} \cdot \Delta t \quad \forall \tau. \quad (5.4)$$

Work and school

Work utility follows a piecewise-linear marginal utility profile around the individual's observed schedule $[t_s, t_e]$ (Joh et al., 2003):

$$\mu_{\text{work}}(\tau) = \begin{cases} \delta - \alpha(t_s - \tau) & \tau < t_s \quad (\text{pre-schedule}), \\ \delta & t_s \leq \tau \leq t_e \quad (\text{on-schedule}), \\ \delta - \beta(\tau - t_e) & \tau > t_e \quad (\text{post-schedule}), \end{cases} \quad (5.5)$$

with $u_{\text{work}} = \mu_{\text{work}}(\tau) \cdot \Delta t$. Here $\delta > 0$ is the on-schedule marginal utility, $\alpha \geq 0$ the earliness penalty (per minute before t_s), and $\beta \geq 0$ the lateness penalty (per minute after t_e); t_s and t_e are injected per individual. School activities are merged into Work at the data layer and share the same specification. The preferred window $[t_s, t_e]$ is an exogenous norm, not each person's own observed hours, so that observed deviation carries the α/β signal; using own-observed hours makes the schedule delay identically zero by construction. An occupation-based norm (the population mean work start/end for the person's occupation code) is a weak proxy for this: it correlates only loosely with an individual's actual departure timing, which understates deviation and attenuates α/β toward zero. Grouping workers instead by joint quantile bins of their own arrival and departure times, and setting the norm to each bin's mean window, keeps the anchor exogenous to the person's schedule while tracking their timing pattern far more closely; this is the anchor used in the baseline estimate (Section 5.6).

Shopping and leisure

Discretionary activity utility is driven by zone attractiveness and opening-hour *availability*, together with a fixed-clock time-of-day base profile. Each zone z has an attractiveness level $X[z]$ (log-normalized retail or leisure floor area; Figure 5.1b) and an opening probability $P_{\text{open}}[z, \tau]$ derived from Google Maps business-hour records:

$$u(z, \tau) = (\beta_1 \cdot X[z] \cdot P_{\text{open}}[z, \tau] + \beta_0(\tau)) \cdot \Delta t, \quad (5.6)$$

where $\beta_0(\tau)$ is a piecewise-linear profile with three knot levels at fixed clock times common to every person: the day start T_{start} , the population-average work-start time t_s^{ref} , and the population-average work-end time t_e^{ref} ; linear interpolation fills between knots and the profile is held flat past t_e^{ref} , leaving P_{open} to carry the evening shape on its own. Because the reference times are fixed at the *population* average rather than each individual's own schedule, $\beta_0(\tau)$ varies only with clock time and stays orthogonal to the schedule-preference parameters δ, α, β : an individual-schedule anchor would let the discretionary profile absorb timing information those parameters are meant to identify. $X[z] \in [0, 1]$ and $P_{\text{open}}[z, \tau] \in [0, 1]$ play distinct roles: P_{open} encodes *when a zone is open* (a feasibility/availability mask), while β_1 scales how strongly attractiveness translates into utility once it is. Shopping and leisure have fully separate parameter sets, $\beta_{1,\text{shop}}$ plus its three $\beta_{0,\text{shop}}(\tau)$ knots, and $\beta_{1,\text{leis}}$ plus its three $\beta_{0,\text{leis}}(\tau)$ knots, so the two activities' time-of-day shapes are estimated independently. P_{open} itself is built from the Google Places API: opening-hours periods for shopping and leisure points of interest in each zone are aggregated into the fraction of points of interest open at each 15-minute step, averaged over Monday to Friday to match the weekday diary, and published as the per-zone lookup table Equation 5.6 reads from.

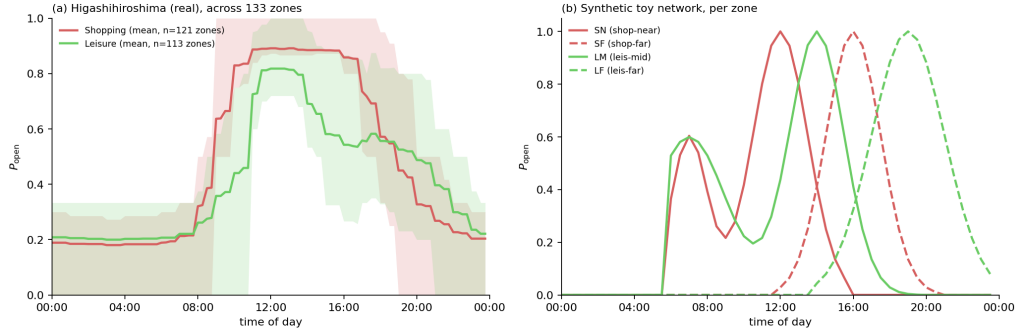


Figure 5.6: P_{open} over the day: Higashihiroshima (left, mean and interquartile range across the 121 of 144 zones with recorded shopping hours and 113 of 144 with recorded leisure hours; zones with no recorded points of interest are excluded rather than assumed closed) against the synthetic toy (right, Section 5.4).

Both real curves rise from a diffuse night floor around 0.2, held up by the all-hours venues counted in the zone total, to a broad daytime plateau peaking near noon, and fall back gradually through the evening, leisure lagging shopping by roughly an hour. The toy's four discretionary zones instead each carry a single Gaussian bump with a distinct peak time and a hard zero outside its window, a caricature of the real shape rather than a reproduction of it, since real venues are never fully closed on aggregate. It is precisely the toy's separated, sharply-timed peaks that let the recovery study of Section 5.4 pin down the attractiveness coefficient β_1 against the base rate β_0 zone by zone.

Travel disutility

The travel disutility for mode m on the (τ, z, z') edge is the mode utility

$$u_{\text{travel}}(m) = \kappa_m + \beta_t \cdot \text{TT}(z, z', m) + \beta_c \cdot \text{cost}(z, z', m), \quad (5.7)$$

where κ_m is the mode-specific constant and β_t, β_c are the time and cost coefficients. This component is specified under two treatments, of which only the first is estimated in this thesis (Section 5.6):

- **Fixed-mode.** The mode coefficients $\{\kappa_m, \beta_t, \beta_c\}$ are fixed at the Higashihiroshima multinomial-logit (MNL) estimates, and a single scalar θ_{travel} rescales the whole mode utility ($1.0 =$ the HH MNL baseline). This is the two-stage treatment of Västberg et al. (2020).
- **Free-mode.** The four mode constants κ_m (car driver, car passenger, walk, bicycle; public transport is the reference) and the coefficients β_t, β_c are estimated *jointly* with the activity parameters; θ_{travel} is then fixed at 1.

Public-transport modes (train, bus) are the reference category, so no separate PT-constant adjustment is required.

Provenance of the fixed-mode coefficients. The Higashihiroshima MNL estimates above are not a published source. They come from an unpublished mode-choice model estimated within the same research group, on the same underlying person-trip survey this thesis’s activity data is drawn from (Section 5.1): a standard multinomial logit over five alternatives (car driver, car passenger, walk, bicycle, public transit as reference), with area-specific constants for Hiroshima city and Higashihiroshima but a single travel-time and travel-cost coefficient pooled across both, estimated separately by day type. This thesis uses the weekday coefficients, $\beta_t = -0.039$ per minute and $\beta_c = -0.088$ per 100 yen (implied value of time 44.13 yen/minute), matching its own weekday scope. The model’s predicted mode shares were checked against the observed shares in that same internal work and matched closely, though only in an internal report, not a citable, peer-reviewed one.

The two treatments trade off differently against the rest of the specification. Fixing the mode coefficients keeps the number of jointly estimated parameters small, at the cost of a single scalar θ_{travel} that must absorb any mismatch between the externally-estimated MNL and the sequential DDCM context in which travel now competes against staying in place, not just against other trips. Because that scalar multiplies the travel-cost term the switching cost c_{change} also operates through (below), the two parameters can share the burden of matching observed trip frequency in ways that are hard to separate under the fixed-mode treatment specifically; estimating mode choice jointly removes this particular source of overload by giving travel cost its own freely-estimated scale. Which treatment better represents the data is an empirical question that the free-mode cell, left for future work (Section 6.3), would answer; this thesis reports only the fixed-mode treatment against the diary (Section 5.6).

Switching cost. The earlier specification carried an activity-switching cost $c_{\text{change}} < 0$ applied at both the departure and arrival edges ($2 c_{\text{change}}$ per trip). When mode is estimated freely it is dropped (pinned to 0), following Västberg et al. (2020), who use no switching cost: real travel time and money already suppress excess trip-making once mode choice is estimated. It is kept (free) only in the robustness runs that hold mode to a single scale.

Parameter summary

Table 5.2 groups the parameters by component and notes which specification each belongs to. Estimated values for the baseline (schedule-preference, fixed-mode, weekday) specification are reported in Section 5.6; calibration values are deliberately not quoted here.

Table 5.2: Utility parameters by component. “Spec” indicates the mode treatment the parameter belongs to, where relevant: FX = fixed mode, FR = free mode. Parameters common to both are marked –.

Component	Symbol	Description	Spec
Work	δ	On-schedule marginal utility (+, per min)	–
	α	Earliness penalty slope (≥ 0)	–
	β	Lateness penalty slope (≥ 0)	–
Discretionary	$\beta_{1,\text{shop/leis}}$	Attractiveness $\times P_{\text{open}}$ coef. (+)	–
	$\beta_{0,\text{shop/leis}}(\tau)$	Fixed-clock 3-knot time-of-day base profile (unrestricted sign)	–
Home	μ_{home}	Reservation floor (+, per min)	–
Switching	c_{change}	Switching cost (≤ 0 , $\times 2/\text{trip}$)	FX only
Mode	θ_{travel}	Travel-utility scale (1.0 = HH MNL)	FX
	κ_m	Mode constants (car/car-pass/walk/bike; PT ref.)	FR
	β_t	Travel-time coefficient (≤ 0)	FR
	β_c	Travel-cost coefficient (≤ 0)	FR

For the welfare counterfactual below, the schedule penalties α (earliness) and β (lateness) are *pinned* from the stated-preference literature (Small, 1982) rather than estimated on the real diary.

5.4 Numerical Experiments

The framework solves the value function exactly and fast. Two further questions decide whether it can carry estimation: does the estimator recover known parameters, and what is lost if the exact solve is replaced by a cheaper approximation? Both are answered here with controlled experiments on a small synthetic network, where the truth is known by construction, before the real Higashihiroshima estimation that follows.

The synthetic network. The experiments run on a small six-zone corridor network, small enough that the value function is solved exactly and the parameters that generate the data are known by construction (Figure 5.7). A home zone anchors the corridor; one destination is a mandatory work zone, reachable by any of the five modes but carrying no shopping or leisure opportunity of its own, and the remaining four are discretionary, two shopping and two leisure, at rising travel distance and with different attractiveness and opening peaks through the day, from a mid-morning shopping zone to a late-evening leisure zone, which separates the attractiveness coefficient from the base utility. Car time grows with distance while an express train runs the corridor and undercuts the car on the far zones, so the mode that wins varies with where a trip goes. The population is a small set of worker and non-worker archetypes, differing in their mandatory-activity sequence and, for workers, their schedule. Every travel time is a multiple of the time step,

so trips land cleanly on the grid. The design is deliberate: the variation in travel distance identifies the travel-cost scale, and the staggered opening peaks identify the discretionary preferences, so a correct estimator should recover the generating parameters.

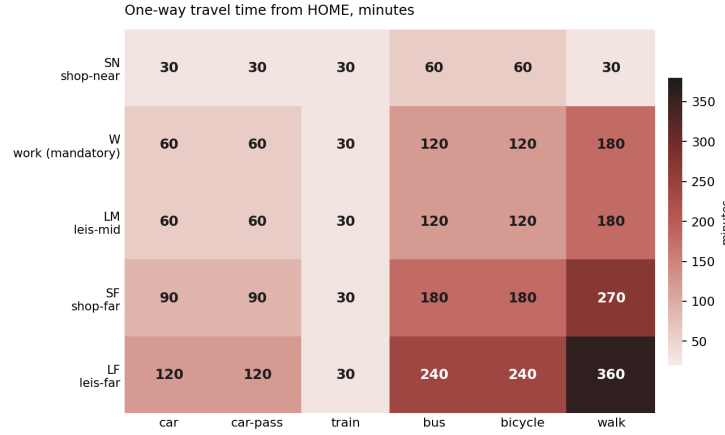


Figure 5.7: The synthetic network behind the recovery and approximation experiments: one-way travel time in minutes from home to each of the five non-home zones, by mode. Car time rises with distance while the express train is flat at thirty minutes regardless of zone, undercutting every other mode on the two far zones (SF, LF); the mandatory work zone (W) and the near leisure zone (LM) sit at the same car distance but are distinct destinations. Each discretionary zone (SN, LM, SF, LF) also carries its own shopping or leisure attractiveness X and opening-hours peak, not shown here (Figure 5.6b plots these directly).

Parameter recovery

The recovery study generates data from a known parameter vector on the small network, re-estimates by the same exact nested fixed-point procedure (Section 2.2), and repeats over many replications. Performance is read off the standard simulation-study measures: *bias*, the mean estimate minus the truth; *coverage*, the share of replications whose confidence interval contains the truth, which should sit near the nominal ninety-five percent; and the ratio of the average model-based standard error to the empirical standard deviation of the estimates, which should sit near one if the standard errors are trustworthy.

The reported experiment is the ten-parameter flat-home schedule-preference specification, matching the fixed-mode treatment the case study uses throughout (Chapter 3). It is run under two conditions at the same realism-calibrated true parameters (chosen so simulated workers spend most of their scheduled day at work, rather than the discretionary-dominated behaviour an earlier calibration produced): a *static*-OD condition, matching the case study’s level-of-service, and a *time-varying*-OD condition, the congestion-premium extension of Section 2.3.1 (Figure 5.8, Table 5.3). Every parameter recovers in both, with coverage at or above the nominal ninety-five percent and low bias; the switching cost c_{change} is the one loosely recovered direction. The model-based standard errors are conservative under the static condition and markedly better calibrated under the time-varying one. The earliness and lateness penalties α and β recover here too, because the data-generating process contains the timing signal by construction; that signal is absent from a static-OD, one-day diary. This does not mean static OD is adequate

on the real diary: there the estimator does not fail loudly by refusing to estimate, but quietly, returning precise, correctly signed penalties whose curvature need not come from the schedule-delay trade-off the specification intends. The synthetic pass certifies the estimator itself, not what α and β mean on the real, static-OD diary.

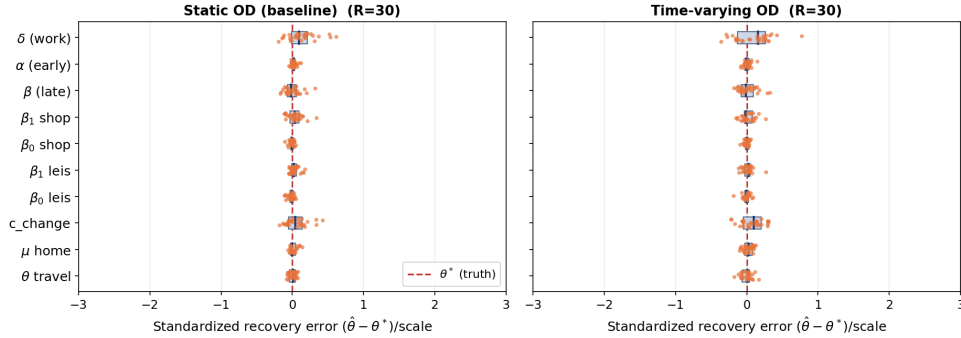


Figure 5.8: Parameter recovery on the small synthetic network, ten-parameter flat-home specification, at the realism-calibrated true parameters. Left: static-OD condition; right: time-varying-OD condition; $R = 30$ replications each ($N = 200$). Each row shows the spread of standardised estimates against the true value (dashed line); every parameter concentrates tightly on the truth, c_{change} the widest.

Table 5.3: Recovery metrics on the synthetic network, static-OD condition ($R = 30$, $N = 200$), ten-parameter flat-home specification. Bias is the mean estimate minus the truth; “Emp. SE” the empirical standard deviation across replications; “Model SE” the mean sandwich standard error; CP the ninety-five-percent coverage probability; APB the absolute percentage bias. Coverage sits at or above nominal throughout; the model-based standard errors are conservative (Model SE $>$ Emp. SE), which is the source of the over-coverage; the time-varying condition improves on it.

Parameter	θ^*	Bias	Emp. SE	Model SE	CP %	APB %
δ (work)	0.200	+0.007	0.010	0.130	100.0	4.4
α (early)	0.0030	+0.0001	0.0002	0.0013	100.0	5.3
β (late)	0.0050	+0.0000	0.0005	0.0029	100.0	8.5
β_1 shop	0.500	+0.011	0.030	0.153	96.7	4.8
β_0 shop	-0.220	-0.004	0.016	0.032	96.7	5.5
β_1 leis	0.400	+0.009	0.018	0.189	93.3	3.6
β_0 leis	-0.420	-0.009	0.021	0.178	96.7	4.1
c_{change}	-0.500	+0.058	0.147	1.556	93.3	24.0
μ home	0.020	+0.001	0.005	0.050	100.0	19.3
θ travel	1.000	+0.002	0.047	0.132	100.0	4.0

Recovery shows that the estimates land on the truth; the profile log-likelihood shows *why* they can. Figure 5.9 slides each parameter across a grid with the others held at the truth and plots the likelihood. For every parameter the likelihood peaks at the true value, which is what identification means: the data respond to the parameter, and a wrong value is penalised. The earliness and lateness penalties peak at the truth like the rest, but with an asymmetric shape, steep below and gentler above, so the data bound how small the penalty can be more tightly than how large. This is the timing signal at work on a network built to

contain it; on a one-day diary the signal is absent and the two penalties are not separately identified.¹

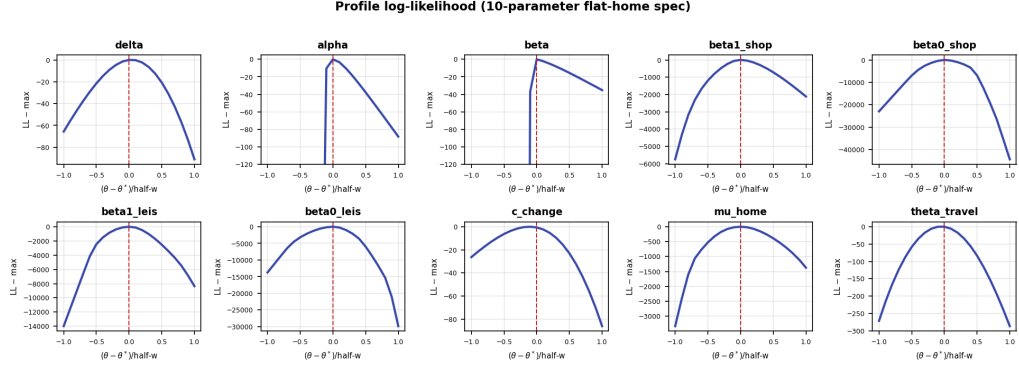


Figure 5.9: Profile log-likelihood for the ten-parameter flat-home specification. Each panel moves one parameter away from the truth (zero on the horizontal axis) with the others held fixed; every panel peaks at the truth. The α and β panels have their vertical axis capped so the curvature near the peak stays visible despite the steep one-sided fall.

Exact, sampling, and reinforcement learning

The exact backward induction is the reference. Two families of approximation make estimation cheaper by avoiding a full solve at every parameter trial. *Sampling of alternatives* draws a set of complete day-paths instead of summing the choice-set log-sum over all continuations, and applies the McFadden correction that restores consistency (McFadden, 1978); this is the route of Västberg et al. (2020), and it does not repeat the backward induction during estimation. *Value estimation by reinforcement learning* replaces the continuation value with a sampled rollout estimate rather than solving it exactly, and without a correction the resulting plug-in likelihood is biased at any finite budget.

The two behave differently because the dynamic likelihood carries a two-layer error, not the single layer a static logit has. The first layer is sampling the choice-set log-sum itself, the layer Dekker et al. (2025) analyses for the static logit. The second, absent from the static case, is that each continuation value is itself produced by backward induction and may be approximated. Sampling of alternatives corrects the first layer and keeps the second exact; reinforcement-learning value estimation leaves the second uncorrected. Figure 5.10 contrasts the three evaluations at a single state.

The experiment compares the sampling distribution of the estimator under each method against the exact estimator on the small network (Figure 5.11). Sampling of alternatives recovers the exact estimator: the estimates concentrate on the same point, with a variance ratio near one, at a fraction of the cost, several times faster at the larger sample sizes. Reinforcement-learning value estimation is biased at finite budget. The importance weights collapse onto a few rollouts, so the estimates miss the exact ones, and the bias recedes only as the budget grows. The lesson is the one the two-layer view predicts: correcting the sampling layer

¹The parameter-recovery study of Figure 5.8 and Table 5.3 uses the realism-calibrated true parameters; the profile likelihood here and the approximation experiment below retain an earlier calibration of the true parameters, which changes neither the identification curvature nor the approximation-error conclusions those two figures illustrate.

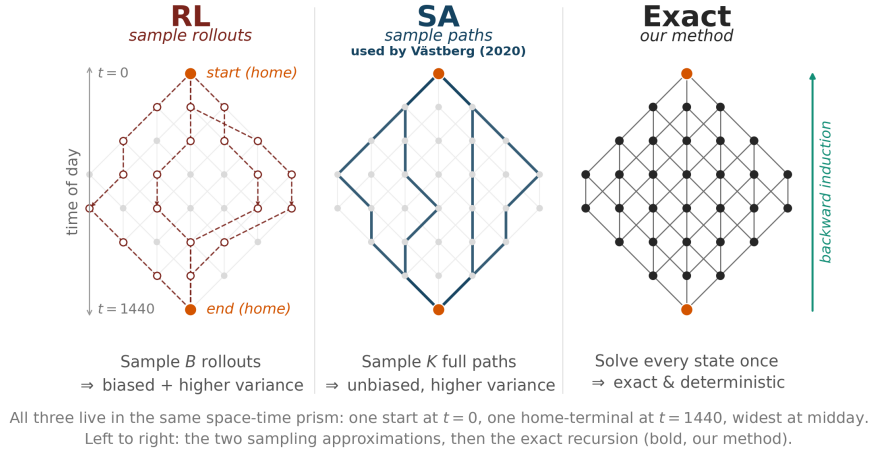


Figure 5.10: Three ways to evaluate the dynamic log-sum at a state, left to right: reinforcement-learning value estimation replaces the continuation value with a sampled estimate and applies no correction; sampling of alternatives sums over a sampled subset with a correction; exact backward induction sums over all continuations.

buys consistency at the price of variance, while leaving the value-approximation layer uncorrected forfeits consistency for speed. The exact estimator stays the reference, and the framework makes it cheap enough that the fallback is seldom needed.

Table 5.4 makes the contrast quantitative. Sampling of alternatives is consistent at every budget: its bias stays below roughly one percent from $K = 5$ upward, and its estimate variance is comparable to the exact sampling-variance floor. Value estimation by reinforcement learning is a different story. Only at the largest budget ($B = 1000$) does it become usable, and even there it is biased (around fourteen percent) and an order of magnitude noisier than the exact estimator; at smaller budgets the plug-in likelihood degenerates, the importance weights collapse onto a handful of rollouts, most replications fail to converge, and the resulting bias and variance are effectively unbounded. This is the two-layer error made visible: correcting the sampling layer (sampling of alternatives) keeps the estimator on the truth, while leaving the value-approximation layer uncorrected (reinforcement learning) does not.

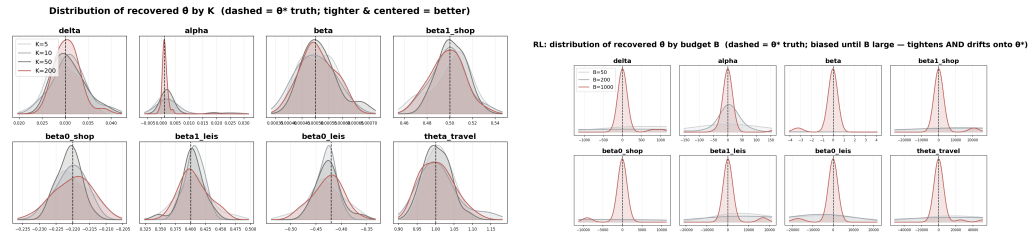


Figure 5.11: Estimator sampling distributions on the small network. Left: sampling of alternatives concentrates on the exact estimate. Right: reinforcement-learning value estimation is biased at finite budget and approaches the exact estimate only as the budget grows.

Table 5.4: Estimator quality on the synthetic network ($R = 30$, $N = 200$). “Bias” is the median absolute relative bias over the well-identified parameters; “Var/Exact” the median ratio of estimate variance to the exact estimator’s, over the same parameters, computed on converged replications (which depresses the sampling ratios below one at low convergence). “ $\gg$ ” marks budgets at which reinforcement learning degenerates: importance weights collapse and bias and variance grow without bound. Sampling of alternatives is consistent at every K ; reinforcement learning is biased at finite budget and usable only as B grows.

Method	Budget	Bias	Var/Exact	Converged
Exact	—	0 (ref.)	$1\times$ (floor)	—
Sampling of alternatives	$K = 5$	0.8%	$\sim 0.6\times$	77%
Sampling of alternatives	$K = 50$	0.5%	$\sim 0.5\times$	77%
Sampling of alternatives	$K = 200$	1.0%	$\sim 0.7\times$	97%
Reinforcement learning	$B = 1000$	14%	$19\times$	40%
Reinforcement learning	$B \leq 200$	$\gg 100\%$	$\gg$ (collapse)	13–43%

5.5 Estimation Problem Setup

The case study estimates the utility parameters θ of Chapter 3 from the Higashihiroshima travel diaries, with the exact framework of Chapter 4 as the inner solver. For a sample of person-days $\{y_n\}_{n=1}^N$ the estimator maximises the log-likelihood

$$\text{LL}(\theta) = \sum_{n=1}^N \sum_k \log P(a_{n,k} \mid s_{n,k}; \theta), \quad (5.8)$$

where the inner sum runs over the decision steps of person n ’s observed day and each step probability is the logit choice probability of Equation (2.3). Because that probability contains the value function $\bar{V}(s; \theta)$, every trial value of θ needs a full backward induction. This is the nested fixed-point structure of Rust (1987), and it is affordable only because the inner solve is the exact, pruned, parallel backward induction of Chapter 4. Estimation is by exact NFXP throughout; the sampling and reinforcement-learning alternatives, and what they cost, were characterised in Section 5.4 above.

The specification matrix. The two utility families of Section 5.3, schedule preferences and time-of-day, can in principle be crossed with the two mode treatments, fixed and free, and with the two day types, giving an $2 \times 2 \times 2$ design space of specifications. This chapter reports the single cell that anchors that space: schedule-preference utility, fixed mode (the transport coefficients held at the external MNL values of Section 5.3), weekday. This baseline is the empirical result the rest of the chapter builds on; the free-mode, time-of-day, and weekend cells are future work (Section 6.3), not a comparison this draft reports.

Optimiser, bounds, and standard errors. The outer problem is solved with L-BFGS-B over the free parameters of the active specification. Economic sign restrictions enter as bounds: the schedule and travel sensitivities (δ , α , β , θ_{travel}) are non-negative; the discretionary base utilities ($\beta_{0,\text{shop}}$, $\beta_{0,\text{leis}}$) and the switching cost (c_{change}) are non-positive, each measured as excess over the home reference; the home level μ_{home} and the attractiveness sensitivities β_1 are unrestricted. The gradient is the analytical score of Section 5.6, so the optimiser never falls back on

finite differences. Standard errors are the robust sandwich form with the score covariance clustered by person, computed at the returned estimate; a checkpoint records the best parameter vector at every improvement, so a long run survives an out-of-memory crash or a disconnect.

5.6 NFXP Implementation

The estimator pairs the outer optimiser with the exact inner solver. For each parameter vector L-BFGS-B proposes, the inner step runs the backward induction of Chapter 4 on each activity-sequence group’s shared graph, which returns both the value function for the likelihood (Equation (5.8)) and, from the analytical gradient sweep of Section 4.4, the score as an observed-minus-expected feature sum; it costs the same order as the value pass and is why the optimiser needs no finite differences. Algorithm 6 states the outer loop: this is the Nested Fixed Point structure of Rust (1987), with Contributions C1–C3 of Chapter 1 supplying the inner solve that makes it affordable at this scale.

Algorithm 6 Nested fixed-point estimation

Require: observed person-days $\{y_n\}$, each assigned to an activity-sequence group $g(n)$ with shared graph G_g (Algorithm 2)

- 1: $\theta \leftarrow \theta_0$
- 2: **repeat**
- 3: **for all** groups g **in parallel do**
- 4: $\bar{V}_g \leftarrow \text{Algorithm 3}(G_g, \theta)$
- 5: $\nabla_\theta \bar{V}_g \leftarrow \text{Algorithm 4}(G_g, \bar{V}_g, \theta)$
- 6: **end for**
- 7: $\text{LL}(\theta) \leftarrow \sum_n \sum_k \log P(a_{n,k} \mid s_{n,k}; \theta)$ using each n ’s $\bar{V}_{g(n)}$ $\triangleright$ Eq. (5.8)
- 8: $\nabla_\theta \text{LL}(\theta) \leftarrow$ observed-minus-expected feature sum, from $\{\nabla_\theta \bar{V}_g\}$
- 9: $\theta \leftarrow \text{L-BFGS-B step}(\theta, \text{LL}(\theta), \nabla_\theta \text{LL}(\theta))$
- 10: **until** convergence
- 11: **return** $\hat{\theta}$, and $\text{LL}, \nabla_\theta \text{LL}$ at $\hat{\theta}$ for standard errors

Convergence on a flat, ridged likelihood is gradual rather than abrupt, and formal non-convergence in the gradient-tolerance sense is normal and harmless: the estimate and its standard errors are computed at whatever vector the optimiser returns, and a near-flat tail does not change them materially. The tolerance is not loosened, and a run is not stopped early to force a result.

Baseline estimate

The baseline schedule-preference, fixed-mode, weekday specification of Section 5.3 is estimated on the full weekday sample: 7,390 valid person-days, 26 activity-sequence groups defined by the joint arrival–departure timing bins (Section 5.3), and 514,261 observed decision steps once the same-zone exclusion of Section 5.1 and the 05:00 day-start restriction (Section 3.3) are applied. These sample counts are a property of the data and the grouping scheme, not of the parameter vector.

The optimizer converged (L-BFGS-B, relative-reduction criterion) at iteration 131, $\log L = -69,965.79$ from an initial $-73,138.39$. Robust sandwich standard errors ($H^{-1}BH^{-1}$, score clustered by person, H from a central finite difference of the analytical gradient) are reported in Table 5.5. Every economic sign restriction holds: $\delta, \alpha, \beta, \theta_{\text{travel}} > 0$ and $c_{\text{change}} < 0$; every parameter is significant at $|t| > 7$. The fitted model reproduces the diary’s aggregate activity-time allocation closely

(McFadden $\rho^2 = 0.913$ against the null model), confirming the estimator, the graph, and the population it was fit to are wired together correctly.

Table 5.5: Baseline estimate: schedule-preference utility, fixed mode, weekday sample. Robust sandwich standard errors, clustered by person.

Symbol	Component	Estimate	Robust SE	Robust t
δ	Work, on-schedule	0.0381	0.0011	34.24
α	Work, earliness	0.0005	0.0000	30.26
β	Work, lateness	0.0003	0.0000	26.93
$\beta_{1,\text{shop}}$	Shop, attractiveness $\times P_{\text{open}}$	0.0184	0.0006	32.71
$\beta_{1,\text{leis}}$	Leisure, attractiveness $\times P_{\text{open}}$	0.0155	0.0004	38.87
$\beta_{0,\text{shop}}(T_{\text{start}})$	Shop, day-start knot	-0.5045	0.0706	-7.15
$\beta_{0,\text{shop}}(t_s^{\text{ref}})$	Shop, work-start knot	0.0156	0.0012	12.70
$\beta_{0,\text{shop}}(t_e^{\text{ref}})$	Shop, work-end knot	0.0107	0.0013	8.15
$\beta_{0,\text{leis}}(T_{\text{start}})$	Leisure, day-start knot	-0.3166	0.0211	-14.99
$\beta_{0,\text{leis}}(t_s^{\text{ref}})$	Leisure, work-start knot	0.0225	0.0011	20.31
$\beta_{0,\text{leis}}(t_e^{\text{ref}})$	Leisure, work-end knot	0.0145	0.0011	13.11
μ_{home}	Home reservation floor	0.0209	0.0011	19.75
c_{change}	Switching cost	-3.6363	0.0252	-144.55
θ_{travel}	Travel-utility scale (1.0 = HH MNL)	0.2210	0.0048	46.19

5.7 Welfare and Policy Impact

The value function at the start of the day, $\bar{V}(s_0; \theta)$, is the expected utility of the best available day and, by the Williams–Daly–Zachary theorem, the welfare measure the policy question needs (Section 2.2). A policy is evaluated as the change $\Delta \bar{V}(s_0) = \bar{V}^{\text{new}}(s_0) - \bar{V}^{\text{base}}(s_0)$, a compensating variation in utility units, computed by re-solving the backward induction under the policy and differencing. No re-estimation is required: the one extra solve is cheap on the pruned graph.

Before turning to a specific policy change, the same quantity can be read directly off the converged model with no counterfactual at all: $\bar{V}(\text{HOME}, z, t=0; \hat{\theta})$, the value of the best day available to a resident of zone z , evaluated at the same $\hat{\theta}$ as Table 5.5 and mapped across every zone in Figure 5.12. This is the same value function the counterfactual differences below, so the map is a baseline welfare surface, not a separate quantity.

The same value function also shows what a household’s car ownership is worth, at no extra estimation cost: car ownership already enters the model as a mask on which travel modes an evaluation context may use (Section 4.5), so re-solving $\bar{V}(\text{HOME}, z, t=0; \hat{\theta})$ once with the car mode excluded, at the same $\hat{\theta}$ and the same network, gives a second accessibility surface directly comparable to the first zone by zone. Figure 5.13 shows both. With a car, accessibility varies substantially across the network (standard deviation 0.283 across the 144 zones), highest in the commercially dense central zones and lowest at the periphery. Without one, accessibility is both lower everywhere and far flatter (standard deviation 0.114): the mean gap is 0.455 logsum units, but it ranges from essentially zero in a handful of peripheral zones up to 0.648 in the zones that benefit most from a car, showing concretely how much of a location’s value depends on car availability rather than the location itself.

The scheduling counterfactual is where the pinned penalties earn their place. α and β are fixed from the stated-preference literature rather than estimated from

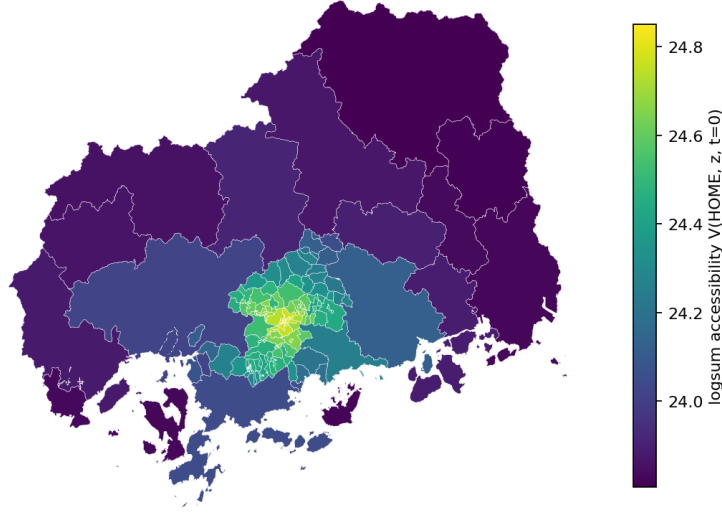


Figure 5.12: $\bar{V}(\text{HOME}, z, t=0; \hat{\theta})$ by home zone, Higashihiroshima, at the converged estimate. Central, commercially dense zones score highest, reflecting shorter trips to the shopping and leisure venues the fitted β_1 coefficients reward.

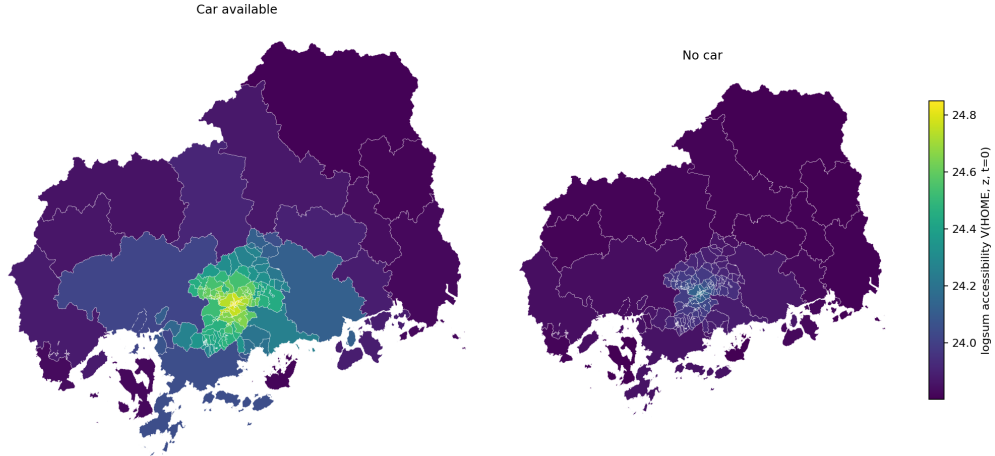


Figure 5.13: $\bar{V}(\text{HOME}, z, t=0; \hat{\theta})$ by home zone, car available (left) versus car excluded (right), same $\hat{\theta}$ and network as Figure 5.12. The central zones' advantage all but disappears without a car: accessibility is both lower and far more uniform across the network.

the diary, and the policy lever, a shift in the preferred arrival time or a change in the cost of travelling at the peak, is applied at those fixed values. Re-solving the day at the pinned parameters gives the welfare change $\Delta \bar{V}(s_0)$, reported per household and aggregated across the population, with a sensitivity band that traces how the result moves as the pinned α and β vary over their plausible stated-preference range. This is the concrete answer to the welfare question that opened the thesis: who gains, who loses, and by how much, in units a planner can use.

The standard errors reported with the estimates are the robust sandwich form $H^{-1}BH^{-1}$, with the score covariance clustered by person; they are computed at the returned estimate whether or not the optimiser reaches its gradient tolerance.

Counterfactual result

The policy lever is a 2-hour later shift of each worker’s own diary WORK window $[t_s, t_e]$ (both endpoints shifted together, so the window’s own duration and the α/β penalty shape around it are unchanged), applied to 150 workers sampled from the real Higashihiroshima diary on the real 144-zone network, at the converged $\hat{\theta}$ of Table 5.5. Because WORK is zone-agnostic in the shared graph (Section 4.6), one graph built once across the sampled workers’ home zones covers the whole exercise; only the per-schedule utility table is recomputed between the base and policy solve, so the 150-worker re-solve costs a handful of extra backward-induction passes, not a new estimation run.

Table 5.6 reports $\Delta\bar{V}(s_0)$ under the free estimate $\hat{\alpha}$ (reported for reference, not as the policy-relevant number) and under three pinned α values, $\alpha = \hat{\beta}/1.5$, $\alpha = \hat{\beta}/2$, and $\alpha = \hat{\beta}/4$, the last being the closest match to Small’s (1982) late-to-early ratio of roughly 4:1 and so the value the sensitivity band is anchored on; β and every other parameter are held at their free estimate in all four rows.

Table 5.6: Compensating variation $\Delta\bar{V}(s_0)$ under the 2-hour WORK-delay counterfactual, $n = 150$ sampled workers, real network. Aggregate is the sum of $\Delta\bar{V}(s_0)$ over the sampled workers, not scaled to the full population.

Condition	α used	Mean $\Delta\bar{V}(s_0)$	Aggregate	Losers	Winners
Free $\hat{\alpha}$ (reference)	0.000464	+0.125	+18.82	0	150
Pinned $\hat{\beta}/1.5$	0.000168	+0.045	+6.74	0	150
Pinned $\hat{\beta}/2$	0.000126	+0.123	+18.40	0	150
Pinned $\hat{\beta}/4$ (Small ratio)	0.000063	+0.554	+83.06	0	150

Every one of the 150 sampled workers gains from the delay, at every pinned α tested and at the free estimate: there are no losers in this sample, only a sensitivity band on the *size* of the gain, not its sign. This is the opposite of the naive expectation that a later, less convenient work schedule should cost someone something, so it is worth being explicit about the mechanism rather than reporting the number without comment. Shifting t_s and t_e together by the same amount does not change the shape of the WORK tent or the total non-work time available in the day; it only moves *which* clock hours that non-work time falls in. The discretionary SHOP/LEISURE profile is a fixed-clock spline, anchored to population-average reference times rather than to each worker’s own schedule (Chapter 3), precisely so that it carries no per-worker schedule information the way an individually-anchored profile would. For the workers in this sample, whose baseline WORK windows start at or before the late-morning reference knot, delaying the start reallocates non-work minutes away from the lower-valued early-evening segment of that profile and toward the higher-valued late-morning segment near the reference knot, at fixed total non-work duration. Every pinned condition shows the same sign because α and β shape the tent around $[t_s, t_e]$, not the discretionary profile the reallocation actually acts on; only the size of β relative to α shifts how much of that reallocated time is spent at WORK’s own margin versus elsewhere, which is why the pinned- α rows move the magnitude without flipping the sign.

This is a real, checkable implication of the fitted specification, not an artefact of one schedule or one parameter draw, since it holds uniformly across 150 diary workers and across the full pinned- α sensitivity band. It is also a level effect of the current discretionary-timing design specifically. A specification with a schedule-anchored rather than fixed-clock discretionary profile, or with duration-dependent

WORK utility, could plausibly reverse it, so it should be read as a property of this fitted model, not a general claim that later work schedules are always welfare-improving. As with the rest of the case study, the result is a per-worker, partial-equilibrium comparison: only the shifted worker’s own day re-solves, and network-wide effects such as other travellers adjusting or congestion responding to the schedule shift are outside the static-OD network’s scope.

A public-transport policy scenario

A second policy, independent of the scheduling counterfactual above, tests a mode-side lever: bus and train travel times across the network are scaled by 0.7, a stylised 30% speed-up standing in for a faster-service investment rather than a specific route proposal. Origin-destination cost is left untouched, so only the two public-transport modes’ relative attractiveness moves. The comparison holds $\hat{\theta}$ fixed and re-solves and re-simulates the same 2,453 sampled residents (the same population, the same seed) once under each network, so every difference below is the policy alone.

Mode share moves toward public transport and walking: bus rises from 1.50% to 2.15% of trip legs, walking from 9.12% to 10.50%, car falls from 82.92% to 81.20%; train, already near-absent on this network, moves little (0.25% to 0.17%). Activity time reallocates correspondingly: leisure time rises from 6.23% to 6.86% of non-travel time, work falls from 19.77% to 19.09%, home and shopping are essentially unchanged. Figure 5.14 shows both.

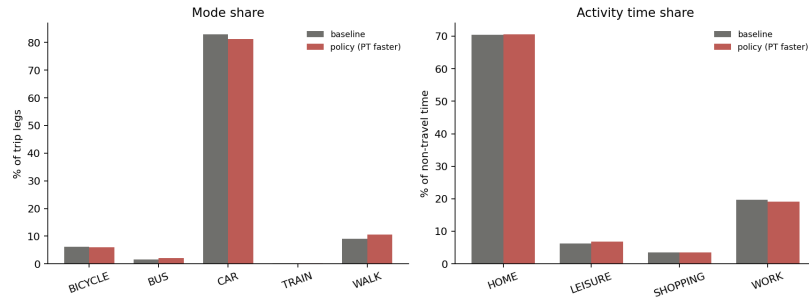


Figure 5.14: Mode share (left) and activity time share (right), baseline versus the public-transport policy, at the converged $\hat{\theta}$, $n = 2,453$ sampled residents.

The same re-solved value function gives a spatial reading of the same policy: $\bar{V}(\text{HOME}, z, t=0; \hat{\theta})$ under the faster-transit network minus the baseline, mapped in Figure 5.15. Every one of the 144 zones gains (mean $\Delta = 0.030$, range $[0.0004, 0.0485]$), and the gain concentrates in the transit-dense zones near the network’s core rather than spreading uniformly, an order of magnitude smaller than the car-ownership gap of Figure 5.13, consistent with a 30% speed-up on a car-dominated network being a smaller lever than having a car at all.

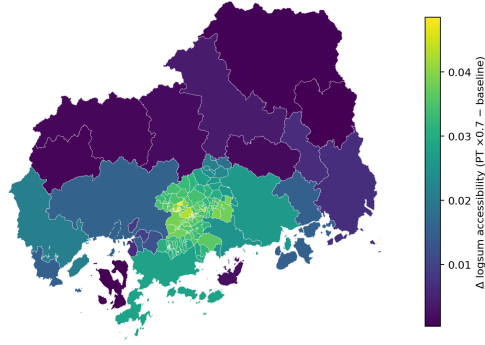


Figure 5.15: $\Delta \bar{V}(\text{HOME}, z, t=0; \hat{\theta})$ under the public-transport policy, by home zone. Every zone gains; the gain concentrates near the transit-dense core.

No re-estimation was needed for either policy: the same converged $\hat{\theta}$ that produced Table 5.5 is re-solved under each network, at the same per-solve cost as the counterfactual above. This is the computational framework’s own claim, Chapter 4, applied directly: exact backward induction is cheap enough that a policy question is answered by re-solving, not by a new estimation run.

Chapter 6

Conclusion

This thesis set out to develop a scalable computational framework for exact-likelihood inference in activity-based dynamic discrete choice models, and to demonstrate that the framework delivers on a real city network. This chapter summarises what was achieved against that objective, states the limitations honestly, and lays out the research programme the work opens.

6.1 Summary of Contributions

The objective of Section 1.3 is met by three contributions, each computational, together with an estimation application built on them and two empirical findings that application produced.

C1: Reachability-based exact state-space reduction. The activity-travel scheduling problem is cast as a finite-horizon optimisation over a directed acyclic graph whose nodes are system states and whose edges are feasible transitions, and a forward breadth-first search under the space-time prism keeps only the states a person can physically reach in a day, cutting the theoretical state space by more than ninety-nine percent (Chapter 4). The two are one package: the DAG framing is what makes the pruned graph solvable in a single non-iterative backward sweep rather than by iterating to a fixed point, and the pruning is what makes that sweep affordable. The reduction is exact, not an approximation: pruned states receive zero probability in the logit model and so leave the value function and every choice probability unchanged.

C2: GPU-parallel exact backward induction. Because the pruned graph has no cycles, every state at a given time step depends only on values already solved at later time steps, so an entire time layer can be solved in one GPU scatter-reduce call rather than state by state (Chapter 4). This takes a full solve from the order of tens of hours to about a hundred seconds, which is what makes exact backward induction affordable inside a maximum-likelihood optimizer rather than merely correct in principle.

C3: Amortized reuse across the population. The shared universal graph makes the cost scale with the number of activity-sequence groups rather than the number of people: one backward induction computation covers an entire population group, so per-agent cost falls toward zero as the population grows (Section 4.6).

The estimation application. Contributions C1–C3 together deliver the objective; whether they actually do is a question only an application can answer. The exact solver is embedded inside a nested fixed-point estimator, with the likelihood gradient computed by a second backward sweep of the same shape as the value sweep (Chapter 5), giving exact maximum likelihood with a gradient that costs the same order as the value solve and no finite-difference noise. The utility specification used for this, time-varying marginal-utility profiles in the temporal-utility-profile tradition (Chapter 3), is the specification adopted for the case study, not a requirement of the framework, which places no restriction on the utility form beyond additive separability across time steps. A parameter-recovery study confirms that the estimator recovers known parameters, and a controlled comparison quantifies what the sampling and reinforcement-learning approximations cost against the exact reference.

Two empirical findings. Beyond the estimates themselves, the application surfaces two further results that are honest about what a one-day diary can and cannot show. The first is an identification finding: the earliness and lateness scheduling penalties are not identified from a one-day diary on a static network, because that data lacks the exogenous variation the penalties are defined against; they are recoverable under the time-varying travel term of Section 2.3.1, on synthetic data where that variation is built in and the true values are known, and are fixed from the stated-preference literature for the welfare analysis on the real, static-network diary. The second is that the value function at the start of the day provides a welfare measure that a policy counterfactual can be read from directly, by re-solving under the policy and differencing (Section 5.7).

6.2 Limitations

The scope of the results is bounded in four ways, each a limitation of the data or the current implementation rather than of the framework.

First, the scheduling penalties are not identified on the revealed-preference diary. This is a property of the data, a one-day observation on a static network, not a defect of the estimator or the model: the travel term already accepts a time-varying extension (Section 2.3.1), demonstrated on synthetic data in Section 5.4, but the Higashihiroshima level-of-service matrix does not supply the time-dependent travel times that extension needs. This is why the welfare counterfactual pins those penalties from external evidence rather than estimating them.

Second, the network is resolved at the zone level, so trips that begin and end in the same zone have no represented movement and are excluded, about a fifth of observed road trips (Section 5.1). This trades within-zone detail for a tractable state space, and a finer network resolution would recover those trips at a higher computational cost.

Third, the modelled extent is a mid-sized city of 144 zones. The contribution is exactness on a full real-city network, not metropolitan coverage; scaling the same machinery to a larger metropolitan network is a matter of compute rather than method, and is left to future work.

Fourth, the fixed-mode specification holds the transport coefficients at external values, so its standard errors do not reflect the uncertainty in those coefficients; the free-mode specification estimates them jointly and is the remedy. The behavioural simulation is a qualitative check on the pipeline, not a formal goodness-of-fit test.

6.3 Future Work

The natural continuations fall into two groups. The first stays within the present framework. The scheduling-penalty identification problem does not have a demonstrated resolution within the present framework as it stands: the time-varying extension of the travel term (Section 2.3.1) is implemented and runs correctly on synthetic data, but the synthetic recovery study does not cleanly show it succeeding where a static network fails, since a synthetic population’s own choice variation can substitute for the missing exogenous force in a way a single real diary day cannot. What the extension needs is a genuine exogenous timing force in the *real* data, which the present level-of-service matrix does not supply. Activity-based traffic microsimulation platforms such as MATSim (Axhausen and ETH Zürich, 2016) are built to supply exactly this: an iterated assignment produces time-of-day travel-time skims that could replace the fixed discomfort premium used here with the network’s own simulated congestion, upgrading it to the physical bottleneck delay the scheduling-delay literature assumes. Whether that genuinely identifies the scheduling penalties from revealed-preference data, rather than merely giving the model a mechanism to use, is precisely the open question this thesis leaves for that coupling to answer: an empirical question about the real network’s actual congestion pattern, not one the present toy network can settle in either direction. A result either way is informative: substantial time-of-day variation in the simulated skims, paired with an estimation that then identifies α and β cleanly, would confirm the extension closes the gap; muted variation, or variation that still fails to identify them, would upgrade the present argument from “the data format cannot supply this force” to an empirically confirmed “the real network does not generate enough of it either”, a more specific limitation than the structural one stated here, and still a legitimate result. Coupling the demand model to a MATSim assignment in this way, an iterative loop in which the DDCM’s predicted demand feeds a traffic assignment whose resulting travel times are fed back into the next round of estimation, is a substantial undertaking in its own right and is left to future work.

Duration-dependent utility, such as a satiation term, could be reopened now that the cost of the extra state dimension has been scoped (Chapter 3); the reachability pruning that dominates the state-space reduction applies to the duration-augmented graph as well. Population heterogeneity could be represented by latent classes solved on the shared graph at little extra cost, a claim the heterogeneity-scaling measurement of Section 5.2 (Figure 5.4) supports directly: solve time is linear in the number of types, so dozens of classes remain affordable.

The second continuation is the doctoral programme the thesis initiates. The exact estimator is a correct but expensive reference, and its inner solve, though far cheaper than a naive one, still grows with the network. Two proven connections point to a faster estimator that keeps the welfare guarantee: the equivalence between logit dynamic discrete choice and maximum-entropy inverse reinforcement learning (Ermon et al., 2015), and the shared graph structure between backward induction and message passing on graphs. The aim is to establish the conditions under which reinforcement-learning and deep-learning machinery can be brought into the estimator without corrupting the welfare measure, and to build an algorithm that is welfare-preserving and faster than exact NFXP. The exact estimator of this thesis is the reference against which that algorithm must be validated, on parameter estimates, standard errors, and welfare figures.

6.4 Concluding Remarks

The activity-based dynamic discrete choice model is the behaviourally realistic tool for transport welfare analysis, and its cost has kept it from being used at the scale policy requires. This thesis shows that the cost is not fundamental: restricting the computation to the states a person can actually reach makes the exact value function affordable on a single machine, without approximation, on a real city network. On that foundation the model can be estimated by exact maximum likelihood, its estimates validated by recovery, its identification limits stated honestly, and its welfare consequences read off directly for policy. What remains is to keep the exactness while going faster, which is the programme this work begins.

Appendix A

Data Description

This appendix reports the Higashihiroshima data as summary tables, computed directly from the processed survey, network, and zone files. It documents what the data contain, not how the pipeline was built or debugged.

A.1 Person-Trip Survey

The behavioural data are the Higashihiroshima person-trip travel survey, a one-day activity-travel diary per respondent, built from the original personal-information roster and the separate weekday and weekend trip files. Respondents who made no trips are kept as all-home person-days, so the model sees the choice between going out and staying in. Table A.1 reports the sample sizes; Table A.2 reports person-level demographics on the full individual roster.

Table A.1: Person-trip survey sample sizes.

Quantity	Count
Individuals (full roster)	10,634
Households	5,288
Weekday person-days (incl. stay-at-home)	9,006
Weekend person-days (incl. stay-at-home)	8,751
Weekday person-days evaluated in the likelihood ^a	7,390
Weekday trip legs (activity = Travel)	18,302

^a After the same-zone exclusion of Section 5.1; see Chapter 5.

Table A.2: Person-level demographics, full individual roster ($N = 10,634$).

Variable	Category	Share
Age	mean 50.1, sd 22.1, range 5–103	—
Gender	Male	50.0%
	Female	48.1%
	Missing	1.9%
Mandatory activity	Has a work/study zone (“worker”)	58.6%
	No mandatory zone (“non-worker”)	41.4%
Vehicle ownership	Car	27.7%
	Motorcycle	2.3%
	Bicycle	17.5%

A.2 OD Level of Service

The network is summarised by an origin-destination level-of-service matrix giving travel time and monetary cost between every pair of the 144 zones ($144^2 = 20,736$ ordered pairs) for each of the active modes. Car times are free-flow network times; other modes come from the corresponding network skims. Table A.3 reports coverage and the time/cost distribution by mode; train service, in particular, covers under half a percent of pairs, consistent with a network where rail only connects a handful of station zones (Section 5.1).

Table A.3: OD level-of-service summary by mode, across all 20,736 zone pairs. Coverage is the share of pairs with a service time recorded; time and cost statistics are computed over covered pairs only.

Mode	Coverage	Mean time (min)	Time range (min)	Mean cost (JPY)
Car	100.0%	37.2	1.7–168.8	426
Bus	99.4%	56.1	0.5–430.9	950
Train	0.4%	51.4	3.0–123.0	712
Walk	100.0%	454.5	4.9–2806.1	—
Bicycle	100.0%	130.7	1.4–806.7	—

A.3 Zone Attractiveness

Zone attractiveness enters the shopping and leisure utilities of Equation (5.6) as $X[z]$, the log-normalized sum of relevant point-of-interest categories per zone (Section 5.3): retail categories for shopping, and dining/entertainment/sports/hospitality categories for leisure. Table A.4 reports the resulting distribution across the 144 zones (visualized as a choropleth in Figure 5.1b); the 144 zones split into the internal Higashihiroshima zones (B -code 1–12, 122 fine-grained zones) and the rest-of-prefecture aggregate (B -code 13, 22 coarse zones), and activity concentrates in the central Saijō area.

Table A.4: Zone attractiveness distribution, $X[z] \in [0, 1]$ across all 144 zones.

	Mean	SD	Min	Max	Zones with zero raw count
$X_{\text{shop}}[z]$	0.240	0.211	0.000	1.000	22 (15.3%)
$X_{\text{leis}}[z]$	0.206	0.215	0.000	1.000	45 (31.3%)

A.4 Mode Ownership and Choice

Car availability determines whether the car-driver mode is feasible, since it requires the household car to be at home (Section 3.2). Table A.2 already reports the ownership shares; Table A.5 adds the observed weekday mode split, over the 18,302 trip legs of Table A.1, which is what the fixed-mode constants of Section 5.3 are calibrated against and the free-mode specification estimates directly.

Table A.5: Observed weekday mode shares, over all trip legs.

Mode	Share of trip legs
Car	57.5%
Walk	12.6%
Car passenger	10.1%
Train	9.4%
Bicycle	5.9%
Bus	1.9%
Other (moped, taxi, motorcycle, etc.)	2.6%

References

- Adler, T. and Ben-Akiva, M. (1979). A theoretical and empirical model of trip chaining behavior. *Transportation Research Part B: Methodological*, 13(3):243–257.
- Axhausen, K. W. and ETH Zürich (2016). *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press.
- Axhausen, K. W., Zimmermann, A., Rindsfuser, G., and Haupt, T. (2002). Observing the rhythms of daily life: A six-week travel diary. *Transportation*.
- Bansal, S., Chen, M., Herbert, S., and Tomlin, C. J. (2017). Hamilton-Jacobi Reachability: A Brief Overview and Recent Advances. arXiv:1709.07523 [eess].
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. arXiv:1502.05767 [cs].
- Berrendorf, R., Makulla, M., and Augustin, S. (2014). Level-Synchronous Parallel Breadth-First Search Algorithms For Multicore and Multiprocessor Systems.
- Besta, M., Marending, F., Solomonik, E., and Hoefer, T. (2017). SlimSell: A Vectorizable Graph Representation for Breadth-First Search. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 32–41, Orlando, FL, USA. IEEE.
- Bowman, J. and Ben-Akiva, M. (2001). Activity-based disaggregate travel demand model system with activity schedules. *Transportation Research Part A: Policy and Practice*, 35(1):1–28.
- Buluc, A. and Madduri, K. (2011). Parallel Breadth-First Search on Distributed Memory Systems. arXiv:1104.4518 [cs].
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to algorithms*. The MIT Press, Cambridge, Massachusetts London, fourth edition edition.
- Dekker, T. (2025). Revisiting McFadden’s correction factor for sampling of alternatives. Working paper – VERIFY metadata from Zotero (venue/volume/pages).
- Dekker, T., Bansal, P., and Huo, J. (2025). Revisiting McFadden’s correction factor for sampling of alternatives in multinomial logit and mixed multinomial logit models. *Transportation Research Part B: Methodological*, 192:103129.
- Doshi, M., Bhabra, M., Wiggert, M., Tomlin, C. J., and Lermusiaux, P. F. J. (2022). Hamilton-Jacobi Multi-Time Reachability. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 2443–2450, Cancun, Mexico. IEEE.
- Dudzík, A. and Veličković, P. (2022). Graph Neural Networks are Dynamic Programmers. arXiv:2203.15544 [cs].

- Eliasson, J., Hultkrantz, L., Nerhagen, L., and Rosqvist, L. S. (2009). The Stockholm congestion – charging trial 2006: Overview of effects. *Transportation Research Part A: Policy and Practice*, 43(3):240–250.
- Ermon, S., Xue, Y., Toth, R., Dilkina, B., Bernstein, R., Damoulas, T., Clark, P., DeGloria, S., Mude, A., Barrett, C., and Gomes, C. (2015). Learning Large-Scale Dynamic Discrete Choice Models of Spatio-Temporal Preferences with Application to Migratory Pastoralism in East Africa. *Proceedings of the AAAI Conference on Artificial Intelligence*, 29(1).
- Fosgerau, M., Frejinger, E., and Karlstrom, A. (2013). A link based network route choice model with unrestricted choice set. *Transportation Research Part B: Methodological*, 56:70–80.
- Frejinger, E., Bierlaire, M., and Ben-Akiva, M. (2009). Sampling of alternatives for route choice modeling. *Transportation Research Part B: Methodological*, 43(10):984–994.
- Guevara, C. A. and Ben-Akiva, M. E. (2013). Sampling of alternatives in multivariate extreme value (MEV) models. *Transportation Research Part B: Methodological*, 48:31–52.
- Hotz, V. J. and Miller, R. A. (1993). Conditional Choice Probabilities and the Estimation of Dynamic Models. *The Review of Economic Studies*, 60(3):497.
- Hägerstrand, T. (1970). WHAT ABOUT PEOPLE IN REGIONAL SCIENCE? *Papers in Regional Science*, 24(1):7–21.
- Joh, C.-H., Arentze, T. A., and Timmermans, H. J. P. (2003). Understanding Activity Scheduling and Rescheduling Behaviour: Theory and Numerical Illustration. In Barlow, M., Boots, B., Okabe, A., and Thomas, R., editors, *Modelling Geographical Systems*, volume 70, pages 73–95. Springer Netherlands, Dordrecht. Series Title: The GeoJournal Library.
- Magnac, T. and Thesmar, D. (2002). Identifying dynamic discrete decision processes. *Econometrica*, 70(2):801–816.
- McCarthy, S., Karlström, A., and Blom Västberg, O. (2025). Activity duration dependent utility in a dynamic scheduling model. *Transportmetrica B: Transport Dynamics*, 13(1).
- McFadden, D. (1978). Modeling the Choice of Residential Location. *Transportation Research Record*, (672).
- Merrill, D., Garland, M., and Grimshaw, A. (2012). Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, pages 117–128, New Orleans Louisiana USA. ACM.
- Merz, F. and Sanders, P. (2014). PReaCH: A Fast Lightweight Reachability Index using Pruning and Contraction Hierarchies. arXiv:1404.4465 [cs].
- Oyama, Y. and Hato, E. (2019). Prism-based path set restriction for solving Markovian traffic assignment problem. *Transportation Research Part B: Methodological*, 122:528–546.

- Palma, A. D. and Fosgerau, M. (2011). Dynamic Traffic Modeling. In De Palma, A., Lindsey, R., Quinet, E., and Vickerman, R., editors, *A Handbook of Transport Economics*. Edward Elgar Publishing.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 [cs.LG].
- Powell, W. B. (2011). *Approximate dynamic programming: solving the curses of dimensionality*. Wiley series in probability and statistics. Wiley, Hoboken, N.J, 2nd ed edition.
- Precup, D., Sutton, R. S., and Singh, S. (2000). Eligibility traces for off-policy policy evaluation. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, pages 759–766.
- Rasouli, S. and Timmermans, H. (2014). Activity-based models of travel demand: promises, progress and prospects. *International Journal of Urban Sciences*, 18(1):31–60.
- Rush, A. M. (2020). Torch-Struct: Deep Structured Prediction Library. arXiv:2002.00876 [cs.CL].
- Rust, J. (1987). Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher. *Econometrica*, 55(5):999.
- Saleem, M., Västberg, O. B., and Karlström, A. (2018). An Activity Based Demand Model for Large Scale Simulations. *Procedia Computer Science*, 130:920–925.
- Sedgewick, R., Van Wyk, C. J., and Sedgewick, R. (2009). *Graph algorithms*. Number Pt. 5 in Algorithms in C++ / Robert Sedgewick. [With C++ consulting by Christopher J. Van Wyk]. Addison-Wesley, Boston, 3. ed., [nachdr.] edition.
- Small, K. A. (1982). The Scheduling of Consumer Activities: Work Trips. *The American Economic Review*, 72:467–479.
- Small, K. A. and Rosen, H. S. (1981). Applied Welfare Economics with Discrete Choice Models. *Econometrica*, 49(1):105.
- Su, C.-l. and Judd, K. l. (2012). Constrained Optimization Approaches to Estimation of Structural Models. *Econometrica*, 80(5):2213–2230.
- Supernak, J. (1992). Temporal utility profiles of activities and travel: Uncertainty and decision making. *Transportation Research Part B: Methodological*, 26(1):60–76.
- Sutton, R. S. and Barto, A. (2014). *Reinforcement learning: an introduction*. Adaptive computation and machine learning. The MIT Press, Cambridge, Massachusetts, nachdruck edition.
- Särkkä, S. and García-Fernández, A. F. (2022). Temporal Parallelisation of Dynamic Programming and Linear Quadratic Control. arXiv:2104.03186 [math.OC].

- Särkkä, S. and García-Fernández, A. F. (2024). Temporal Parallelisation of the HJB Equation and Continuous-Time Linear Quadratic Control. arXiv:2212.11744 [math.OC].
- Tithi, J. J., Fogel, Y., and Chowdhury, R. (2022). An Optimal Level-synchronous Shared-memory Parallel BFS Algorithm with Optimal parallel Prefix-sum Algorithm and its Implications for Energy Consumption. arXiv:2209.08764 [cs].
- Vickrey, W. S. (1969). Congestion Theory and Transport Investment. *The American Economic Review*, 59(2):251–260.
- Västberg, O. B., Karlström, A., Jonsson, D., and Sundberg, M. (2020). A Dynamic Discrete Choice Activity-Based Travel Demand Model. *Transportation Science*, 54(1):21–41.
- Zhao, J., Yang, L., Zhang, H., He, Q., and Ding, T. (2026). From Sequential to Parallel: Reformulating Dynamic Programming as GPU Kernels for Large-Scale Stochastic Combinatorial Optimization. arXiv:2602.05179 [math.OC].
- Zhou, H., Dorsman, J., Snelder, M., Romph, E. D., and Mandjes, M. (2019). GPU-based Parallel Computing for Activity-based Travel Demand Models. *Procedia Computer Science*, 151:726–732.
- Ziebart, B. D., Maas, A., Bagnell, J. A., and Dey, A. K. (2008). Maximum Entropy Inverse Reinforcement Learning. *AAAI Press*.